COMSATS University Islamabad, Vehari Campus, Pakistan

Department of Software Engineering

Course: CSC336 Advance Web Technologies

Expense Manager

MERN Stack Web Application · Full-Stack · AI-Powered

GitHub Repository:
https://github.com/rabeelfatma/advance_web_Rabeel_032

A Semester Project Report Submitted in Partial Fulfillment of the Requirements for the Degree of

Bachelor of Science in Software Engineering

Submitted By

Rabeel Fatima SP23-BSE-032

Instructor

Ma'am Yasmeen Jana

Semester: Spring 2026

Executive Summary

Expense Manager is a full-stack MERN web application developed as part of the Advance Web Technologies (CSC336) course at COMSATS University Islamabad, Vehari Campus. The application enables users to track their daily income and expenses, set budgets, monitor financial health, and receive AI-powered personalized financial advice through Google Gemini AI integration.

The system is built using React.js on the frontend and Node.js with Express.js on the backend, with MongoDB as the primary database managed through the Mongoose ODM. User authentication is handled securely using JWT (JSON Web Tokens) with bcrypt-hashed passwords. Password validation enforces a minimum security policy requiring at least 2 alphabets, 2 numbers, and 1 special character.

The application provides more than 30 distinct features across six pages: Landing Page, Signup, Login, Forgot Password, Dashboard, and Analytics. Key features include real-time transaction management with full CRUD operations, a smart notification system, a financial health score widget, budget progress tracking, top expense categories analysis, dark/light mode toggle, PDF report export using jsPDF, and interactive charts (Line, Pie, Bar) powered by Recharts.

The RESTful API exposes nine endpoints across four resource groups (Auth, Transaction, AI Advisor, Contact), all secured with JWT middleware. The AI Financial Advisor integrates with Google Gemini AI to provide personalized financial tips based on the user's spending patterns.

Abbreviations

Abbreviation	Full Form
MERN	MongoDB, Express.js, React.js, Node.js
API	Application Programming Interface
REST	Representational State Transfer
JWT	JSON Web Token
CRUD	Create, Read, Update, Delete
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
ODM	Object Document Mapper
UI	User Interface
PDF	Portable Document Format
AI	Artificial Intelligence
URL	Uniform Resource Locator
SRS	Software Requirements Specification
BSE	Bachelor of Science in Software Engineering
CUI	COMSATS University Islamabad
TC	Test Case
FR	Functional Requirement

Contents

Chapter 1

Introduction

1.1 Introduction

Expense Manager is a full-stack MERN web application that enables users to track their daily income and expenses, set budgets, monitor financial health, and receive AI-powered personalized financial advice. The project integrates a React.js frontend, Node.js + Express.js backend, MongoDB database, and Google Gemini AI. Authentication is handled via JWT tokens with bcrypt password hashing. Password validation enforces a security policy requiring at least 2 alphabets, 2 numbers, and 1 special character.

1.2 Student Information

Field	Details
Student Name	Rabeel Fatima
Registration #	SP23-BSE-032
Subject	Advance Web Technologies (CSC336)
Instructor	Ma'am Yasmeen Jana
University	COMSATS University Islamabad — Vehari Campus
Semester	7th Semester — BS Software Engineering
GitHub	https://github.com/rabeelfatma/advance_web_Rabeel_032

1.3 Tools and Technologies

Tool / Technology	Version	Purpose
React.js	Latest	Frontend UI – pages, components, routing, state management
React Router	Latest	Client-side routing between pages
Axios	Latest	HTTP client for REST API calls from frontend to backend
Recharts	Latest	Chart library for Line, Pie, and Bar charts

Tool / Technology	Version	Purpose
Node.js	Latest	JavaScript runtime for backend server
Express.js	Latest	Web framework for building RESTful APIs
MongoDB	Latest	NoSQL database for storing users, transactions, and contacts
Mongoose	Latest	ODM for defining schemas and interacting with MongoDB
bcryptjs	Latest	Password hashing for secure storage
jsonwebtoken	Latest	JWT generation and verification for stateless authentication
Google Gemini AI	API	AI-powered financial advice based on user transaction data
jsPDF	CDN	Client-side PDF generation for transaction reports
Vite	Latest	Frontend build tool and development server
dotenv	Latest	Environment variable management (.env file)
nodemon	Latest	Auto-restart backend server during development
cors	Latest	Enable Cross-Origin Resource Sharing
Visual Studio Code	Latest	IDE used for development

1.4 Pages and Routes

Route	Page	Description
/	LandingPage.jsx	Marketing page with hero, features, stats, contact form, footer
/signup	Signup.jsx	User registration with validation and password strength policy
/login	Login.jsx	Email/password authentication – returns JWT token
/forgot-password	ForgotPassword.jsx	Direct password reset without OTP
/dashboard	Dashboard.jsx	Main transaction manager with widgets, AI advisor, and PDF export
/analytics	AnalysisPage.jsx	Financial analytics with Line, Pie, Bar charts and time filters

1.5 Project Features

#	Feature	Description	File
1	User Signup	New user registration with validation and password policy	Signup.jsx
2	User Login	Email/password login – returns JWT token	Login.jsx
3	Password Reset	Reset password directly without OTP	ForgotPassword.jsx
4	Password Validation	Min 2 alphabets, 2 numbers, 1 special character	Signup.jsx
5	Show/Hide Password	Toggle password visibility on all auth pages	Login.jsx
6	Landing Page	Full marketing page with all sections	LandingPage.jsx
7	Smooth Scroll	Navbar links scroll to sections without reload	LandingPage.jsx
8	Contact Form	Send message via API with status feed-back	LandingPage.jsx
9	Add Transaction	Title, Amount, Type, Category, Date form	Dashboard.jsx
10	Edit Transaction	Pre-fill form – update via PUT API	Dashboard.jsx
11	Delete Transaction	Confirm dialog – DELETE API call	Dashboard.jsx
12	Real-time Search	Case-insensitive title search	Dashboard.jsx
13	Category Filter	Filter by transaction category	Dashboard.jsx
14	Date Range Filter	Start & end date custom range filter	Dashboard.jsx
15	Quick Time Filters	ALL / DAILY / MONTHLY / YEARLY tabs	Dashboard.jsx
16	Live Summary Bar	Income, Expense, Balance (real-time)	Dashboard.jsx
17	Smart Notifications	Auto-generated bell notifications (3 types)	Dashboard.jsx
18	Dark Mode	Full app dark/light theme toggle	Dashboard.jsx
19	Largest Expense Widget	Auto-finds the biggest single expense	Dashboard.jsx
20	Financial Health Score	0–100 score based on savings rate	Dashboard.jsx
21	Budget Progress Bar	Green <80%, Orange <100%, Red = exceeded	Dashboard.jsx
22	Top Categories Widget	Top 4 expense categories with progress bars	Dashboard.jsx
23	AI Financial Advisor	Gemini AI personalized financial tips	Dashboard.jsx
24	Budget Limit Setting	User sets custom budget – live updates all widgets	Dashboard.jsx
25	PDF Export	Full transaction report download via jsPDF	Dashboard.jsx
26	Logout	Confirm dialog – clear JWT – redirect to login	Dashboard.jsx
27	Line Chart	Income vs Expense trend over time	AnalysisPage.jsx
28	Pie Chart	Budget distribution donut chart	AnalysisPage.jsx
29	Bar Chart	Income vs Expense bar comparison	AnalysisPage.jsx
30	Analytics Filters	Daily/Monthly/Yearly on all 3 charts	AnalysisPage.jsx

1.6 Project Flow

Step 1. User opens the Landing Page – reads features, can contact via the contact form.

Step 2. User clicks Sign Up – fills form with validation (password: min 2 alphabets, 2 numbers, 1 special character) – account created.

Step 3. User logs in with email and password – JWT token stored in localStorage – Dashboard opens.

Step 4. User adds transactions (income/expense) – saved in MongoDB via backend.

Step 5. Dashboard shows live summary: Balance, Income, Expense.

Step 6. User can search, filter by category and date range, edit, and delete transactions.

Step 7. Smart notifications auto-generated based on budget usage.

Step 8. AI Advisor button – Gemini AI analyzes data – gives personalized financial tips.

Step 9. Export PDF – full transaction report downloaded via jsPDF.

Step 10. Analytics page – Line, Pie, Bar charts with time filters (Daily/Monthly/Yearly).

Step 11. User logs out – token cleared – redirected to login.

1.7 Folder Structure

1.7.1 Backend Structure

Path	**Description**
backend/models/User.js	MongoDB schema for user accounts (name, email, hashed password)
backend/models/Transaction.js	MongoDB schema for transactions (title, amount, type, category, date, userId)
backend/controllers/authController.js	signup(), login(), forgotPassword() – auth logic
backend/controllers/transactionController.js	getTransactions(), addTransaction(), updateTransaction(), deleteTransaction()
backend/routes/authRoutes.js	POST /api/auth/signup, /api/auth/login, /api/auth/forgot-password
backend/routes/transactionRoutes.js	GET/POST /api/transactions, PUT/DELETE /api/transactions/:id
backend/routes/contactRoutes.js	POST /api/contact/send
backend/routes/aiAdvisor.js	POST /api/ai-advice – Gemini AI integration
backend/.env	MONGO_URI, JWT_SECRET, OPENROUTER_API_KEY, PORT
backend/server.js	Main Express app entry point – middleware, routes, MongoDB connection
backend/package.json	Backend dependencies (express, mongoose, cors, dotenv, bcryptjs, jsonwebtoken)

1.7.2 Frontend Structure

Path	Description
frontend/src/pages/LandingPage.jsx	Marketing landing page
frontend/src/pages/Login.jsx	Login page with JWT authentication
frontend/src/pages/Signup.jsx	Registration with password validation
frontend/src/pages/ForgotPassword.jsx	Direct password reset page
frontend/src/pages/Dashboard.jsx	Main dashboard – transactions, widgets, AI advisor
frontend/src/pages/AnalysisPage.jsx	Financial analytics with charts
frontend/src/api.js	Axios instance – base URL + JWT Authorization header interceptor
frontend/src/App.jsx	React Router – all route definitions
frontend/src/App.css	Global application styles
frontend/vite.config.js	Vite configuration file
frontend/package.json	Frontend dependencies (react, react-router-dom, axios, recharts)

1.8 System Architecture

The Expense Manager follows a three-tier client-server architecture:

Presentation Layer: React.js frontend with React Router for client-side navigation. All API calls are made through Axios with a centralized JWT interceptor in api.js.

Application Layer: Node.js + Express.js backend exposing RESTful APIs. JWT token verification protects all transaction and AI endpoints. Controllers handle business logic for auth, transactions, AI advice, and contact.

Data Layer: MongoDB database with Mongoose schemas. Separate collections for users, transactions, and contacts. Each transaction is linked to a user via userId to ensure data isolation.

1.9 Objectives of the Proposed System

BO-1: Provide users with a comprehensive personal finance tracking system covering income, expenses, budgets, and savings.

BO-2: Integrate Google Gemini AI to deliver context-aware, personalized financial advice based on actual user spending patterns.

BO-3: Implement a secure JWT-based authentication system with bcrypt password hashing and enforced password validation policy.

BO-4: Deliver a rich analytics dashboard with interactive charts (Line, Pie, Bar) and real-time financial widgets.

BO-5: Enable users to export transaction reports as PDF documents for offline record-keeping.

1.10 Module Descriptions

1.10.1 Module 1: Authentication Module

The Authentication module handles user registration, login, and password reset. During signup, the system validates credentials and enforces a password policy requiring at least 2 alphabets, 2 numbers, and 1 special character. Passwords are hashed using bcryptjs before storage. On login, the system verifies the email and password, then issues a JWT token stored in the browser's localStorage.

Functional Requirements:

FR-1: The system shall allow users to register with validated credentials and password strength enforcement.

FR-2: The system shall authenticate users via email and password and issue a JWT token.

FR-3: The system shall allow users to reset their password directly without OTP verification.

1.10.2 Module 2: Transaction Management Module

The core module allowing authenticated users to add, edit, delete, and view financial transactions. Each transaction contains a title, amount, type (income or expense), category, and date. The dashboard provides real-time summary widgets, search, filter by category and date range, and quick time tabs (ALL/DAILY/MONTHLY/YEARLY).

Functional Requirements:

FR-1: The system shall allow authenticated users to create, read, update, and delete transactions.

FR-2: The system shall display real-time balance, total income, and total expense summaries.

FR-3: The system shall support filtering transactions by title search, category, date range, and time period.

1.10.3 Module 3: Analytics and Insights Module

Provides visual representation of financial data through three chart types: a Line Chart showing income vs expense trends over time, a Pie Chart showing budget distribution, and a Bar Chart comparing total income and expense. Charts support Daily, Monthly, and Yearly time filters.

Functional Requirements:

FR-1: The system shall render Line, Pie, and Bar charts from transaction data using Recharts.

FR-2: The system shall support time-based filtering (Daily/Monthly/Yearly) for all charts.

FR-3: The system shall calculate and display total savings, savings rate, and top

expense category.

1.10.4 Module 4: AI Financial Advisor Module

Integrates Google Gemini AI (via OpenRouter) to provide users with personalized financial tips. When the user clicks “Get AI Advice”, the system sends the user’s financial summary to the AI endpoint. The response returns structured cards with icons, titles, and actionable tips.

Functional Requirements:

FR-1: The system shall send user financial data to the Gemini AI endpoint and return structured advice cards.

FR-2: The system shall display a skeleton loading animation during AI response processing.

FR-3: The AI endpoint shall be protected by JWT token verification.

1.10.5 Module 5: Widgets and Dashboard Module

Provides five real-time financial widgets: Largest Expense, Financial Health Score (0–100), Budget Progress Bar (color-coded), Top 4 Expense Categories, and Smart Notifications (bell icon with badge count).

Functional Requirements:

FR-1: The system shall compute and display real-time financial health metrics from transaction data.

FR-2: The system shall generate smart notifications when budget thresholds are reached.

FR-3: The system shall allow users to set and update a custom budget limit.

1.10.6 Module 6: Contact and Landing Page Module

A full marketing page featuring a sticky navbar with smooth scroll, hero section with CTA buttons, stats bar, features grid with 6 cards, About section, and a contact form. The contact form posts to /api/contact/send with status feedback.

Functional Requirements:

FR-1: The system shall provide a marketing landing page accessible without authentication.

FR-2: The system shall allow visitors to submit a contact message via a form with real-time status feedback.

FR-3: The navbar shall support smooth scrolling to all page sections.

Chapter 2

Implementation

2.1 Development Environment

The Expense Manager was developed using Visual Studio Code as the primary IDE. The backend runs on Node.js with Express.js and connects to a local MongoDB instance. The frontend is built with Vite + React.js and runs on port 5173.

Environment Variables (.env):

MONGO_URI=mongodb://127.0.0.1:27017/expense_tracker

JWT_SECRET=secret123

OPENROUTER_API_KEY=sk-or-v1-... (Gemini AI via OpenRouter)

PORT=5000

2.2 Backend Implementation

2.2.1 Database Models

User Model (User.js): Fields: name, email (unique), password (bcrypt hashed), createdAt. Validation enforces email format, unique email, and bcrypt hashing before save.

Transaction Model (Transaction.js): Fields: title, amount, type (income/expense), category, date, userId (ref: User). The userId field links each transaction to the authenticated user ensuring per-user data isolation.

2.2.2 Controllers

authController.js

signup(): Validates fields, checks for duplicate email, hashes password with bcryptjs, saves User to MongoDB, returns success.

login(): Finds user by email, compares password with bcryptjs, generates JWT token with jsonwebtoken, returns token.

forgotPassword(): Finds user by email, hashes new password, updates document in DB, returns success.

transactionController.js

getTransactions(): Finds all transactions where userId matches logged-in user – returns array.

addTransaction(): Validates fields, creates Transaction with userId from JWT, saves and returns new document.

updateTransaction(): Finds by id + userId, updates fields, saves and returns updated document.

deleteTransaction(): Finds by id + userId, removes from DB, returns success message.

2.3 External APIs and SDKs

API / SDK	Purpose	Module
Express.js	Builds RESTful APIs for communication between frontend and backend	Auth, Transaction, AI, Contact
bcryptjs SDK	Hashes user passwords before storage and compares hashes during login	Authentication Module
jsonwebtoken SDK	Generates and verifies JWT tokens for stateless authentication	Authentication Module
Google Gemini AI	Provides personalized financial advice based on user spending data	AI Financial Advisor
jsPDF (CDN)	Client-side PDF generation for transaction reports	Dashboard – PDF Export
Recharts	React chart library for Line, Pie, and Bar charts	Analytics Module

2.4 RESTful API – Complete Reference

The Expense Manager exposes 9 REST API endpoints across 4 resource groups. All endpoints return JSON. Transaction and AI endpoints require a valid JWT token in the Authorization: Bearer <token> header.

#	Method	Endpoint	Description
1	POST	/api/auth/signup	Register new user. Body: {name, email, password}. Returns 201 Created.
2	POST	/api/auth/login	Login – returns JWT token. Body: {email, password}. Returns 200 OK with token.

#	Method	Endpoint	Description
3	POST	/api/auth/forgot-password	Reset password directly. Body: {email, newPassword}. Returns 200 OK.
4	GET	/api/transactions	Get all transactions for logged-in user. JWT required. Returns 200 OK with array.
5	POST	/api/transactions	Create new transaction. JWT required. Body: {title, amount, type, category, date}. Returns 201.
6	PUT	/api/transactions/:id	Update existing transaction. JWT required. Body: {title, amount, type, category, date}.
7	DELETE	/api/transactions/:id	Delete a transaction by ID. JWT + :id param. Returns 200 OK.
8	POST	/api/ai-advice	Get Gemini AI financial advice. JWT required. Body: financial summary object.
9	POST	/api/contact/send	Send contact form message. Body: {name, email, subject, message}. Returns 200 OK.

2.4.1 HTTP Methods Used

Method	Usage in Expense Manager
GET	Retrieve data – no request body – used for fetching all user transactions (/api/transactions)
POST	Send data to create a new resource – used for signup, login, add transaction, AI advice, contact
PUT	Update an existing resource – used for editing a transaction (/api/transactions/:id)
DELETE	Remove a resource – used for deleting a transaction (/api/transactions/:id)

2.4.2 HTTP Status Codes

Status Code	Meaning
200 OK	Request successful. Data returned or operation completed.
201 Created	New resource created successfully (signup, add transaction).
400 Bad Request	Missing or invalid fields in request body.
401 Unauthorized	JWT token missing, invalid, or expired.

Status Code	Meaning
404 Not Found	Resource (user/transaction) not found in database.
409 Conflict	Duplicate email – user already registered.
500 Internal Error	Server/database error – unexpected failure.

2.4.3 Authentication Header

Field	Value
Header Name	Authorization
Header Value	Bearer <token>
Token Storage	localStorage.getItem('token') on frontend (set via api.js Axios interceptor)
On Failure	401 Unauthorized returned if token missing or expired

2.5 Installation Commands

2.5.1 Backend Setup

Step	Command
Initialize project	npm init -y
Install dependencies	npm install express mongoose cors dotenv bcryptjs jsonwebtoken
Install dev tools	npm install -D nodemon
Run development server	npm run dev or node server.js

2.5.2 Frontend Setup

Step	Command
Create Vite project	npm create vite@latest expense-manager ---template react
Install base packages	npm install
Install dependencies	npm install react-router-dom axios recharts
Run development server	npm run dev
Build for production	npm run build

2.6 Frontend – Page by Page Implementation

2.6.1 Landing Page (LandingPage.jsx)

The Landing Page is the public-facing marketing page. It features a sticky navbar with smooth-scroll links (Home, Features, About, Contact) and Login/Sign Up buttons. The hero section displays the headline “Take Control of Your Finances” with a custom SVG illustration. A stats bar shows 10K+ Users, 500K+ Transactions, 99% Uptime, and 100% Secure. The features grid presents 6 cards. The contact form POSTs to /api/contact/send with real-time status feedback.

2.6.2 Signup Page (Signup.jsx)

The Signup page provides a registration form with Name, Email, Password, and Confirm Password fields. Password validation enforces the security policy: minimum 2 alphabets, 2 numbers, and 1 special character. Both password fields include a Show/Hide toggle. The form POSTs to /api/auth/signup.

2.6.3 Login Page (Login.jsx)

The Login page provides email and password fields with a Show/Hide password toggle. The form POSTs to /api/auth/login. On success, the JWT token is stored in localStorage and the user is navigated to /dashboard. Navigation links go to Forgot Password (/forgot-password) and Signup (/signup).

2.6.4 Forgot Password Page (ForgotPassword.jsx)

The Forgot Password page allows direct password reset without OTP. Fields include Email, New Password, and Confirm Password. One eye icon toggle controls visibility for both password fields. A client-side match check alerts the user if passwords do not match. The form POSTs to /api/auth/forgot-password with {email, newPassword}.

2.6.5 Dashboard (Dashboard.jsx)

The Dashboard is the main application page and the most feature-rich component. It contains:

Summary Bar: Live Balance (Income minus Expense), Total Income, Total Expense

Widget Cards: Largest Expense (reduce() finds max), Financial Health Score (0–100: Excellent/Good/Fair/Poor), Budget Progress Bar (Green <80%, Orange <100%, Red = exceeded)

Top Categories Widget: Top 4 expense categories with proportional progress bars

AI Financial Advisor: Gemini AI button sends financial context and returns advice cards with icons, titles, and tips. Skeleton loading animation shown during processing.

Transaction Form: Add (POST) and Edit (PUT) modes with fields: Title, Amount, Type, Category, Date

Filter System: Real-time title search, category dropdown, date range pickers, quick tabs (ALL/DAILY/MONTHLY/YEARLY)

Smart Notifications: Bell icon with red badge count; alerts for budget exceeded, 80% warning, and last entry

Dark Mode: Sun/Moon toggle covers page background, cards, inputs, widgets, and AI advisor

Budget Limit Setting: User enters custom budget – all widgets update live

PDF Export: jsPDF generates transactions_report.pdf with full history

Logout: Confirm dialog clears JWT from localStorage and redirects to login

2.6.6 Analytics Page (AnalysisPage.jsx)

The Analytics page provides three interactive charts built with Recharts:

Line Chart: Income vs Expense trend over time – green line for income, red line for expense. X-axis shows HH:MM for daily, date for monthly/yearly.

Pie Chart (Donut): Budget distribution showing Income vs Expense proportions.

Bar Chart: Side-by-side Income vs Expense bar comparison.

Stats cards show: Total Savings (income minus expense) with savings rate %, Top Expense Category (highest spend), and Total Transactions count. Time filter tabs (ALL/DAILY/MONTHLY/YEARLY) apply to all charts simultaneously.

2.7 Database (MongoDB)

The Expense Manager uses MongoDB with the database name expense_tracker. Three collections are maintained:

users: Stores user accounts with name, email, and bcrypt-hashed password.

transactions: Stores financial records with title, amount, type, category, date, userId, createdAt, and updatedAt.

contacts: Stores contact form submissions with name, email, subject, message, and createdAt.

2.8 Deployment

Component	Platform	URL / Details
Backend	Local / Node.js	http://localhost:5000 – Express.js server
Frontend	Local / Vite	http://localhost:5173 – React.js dev server
Database	Local / MongoDB	mongodb://127.0.0.1:27017/expense_tracker
GitHub	GitHub	https://github.com/rabeelfatma/advance_web_Rabeel_032

Chapter 3

Testing

This chapter presents the testing strategies applied to the Expense Manager application. Testing ensures that all functional requirements defined in earlier chapters are satisfied across all modules.

Module	TC ID	FR	Description	Input / Action	Expected Result	Status
M1	TC-1.1a	FR-1.1	Successful Signup	Valid name, email, password (meets policy)	Account created; user redirected to /login	Pass
M1	TC-1.1b	FR-1.1	Weak Password Signup	Password without special character	System shows validation error	Pass
M1	TC-1.1c	FR-1.1	Duplicate Email	Register with already used email	System returns 409 Conflict	Pass
M1	TC-1.2a	FR-1.2	Successful Login	Valid email + valid password	JWT token stored; user redirected to /dashboard	Pass
M1	TC-1.2b	FR-1.2	Invalid Login	Valid email + wrong password	Error message “Invalid credentials”	Pass
M1	TC-1.2c	FR-1.2	Empty Field Validation	Leave email or password blank	System prompts “Please fill in this field”	Pass
M1	TC-1.3a	FR-1.3	Forgot Password Link	View login page	Reset link visible and clickable	Pass

Module	TC ID	FR	Description	Input / Action	Expected Result	Status
M1	TC-1.3b	FR-1.3	Password Reset	Submit email + new password	Password updated; redirected to /login	Pass
M1	TC-1.3c	FR-1.3	Password Mismatch	New password ≠ confirm password	Alert: "Passwords do not match" shown	Pass
M1	TC-1.4a	FR-1.4	Password Hashing	Check users collection in MongoDB	Password field shows bcrypt hash	Pass
M2	TC-2.1a	FR-2.1	Add Transaction	Fill form with valid data	Transaction saved; summary updated	Pass
M2	TC-2.2a	FR-2.2	Edit Transaction	Click Edit on existing transaction	Form pre-filled; update successful	Pass
M2	TC-2.3a	FR-2.3	Delete Transaction	Click Del on existing transaction	Transaction removed from list and DB	Pass
M3	TC-3.1a	FR-3.1	Line Chart	View Analytics page	Income vs Expense line chart rendered	Pass
M3	TC-3.1b	FR-3.1	Pie Chart	View Analytics page	Budget distribution pie chart rendered	Pass
M3	TC-3.1c	FR-3.1	Bar Chart	View Analytics page	Income vs Expense bar chart rendered	Pass
M4	TC-4.1a	FR-4.1	AI Advice	Click "Get AI Advice"	Gemini AI returns Structured Advice Cards	Pass

Module	TC ID	FR	Description	Input / Action	Expected Result	Status
M5	TC-5.1b	FR-5.1	Budget Progress	Set budget limit	Budget bar updates; color changes at thresholds	Pass
M5	TC-5.1c	FR-5.2	Smart Notifications	Exceed budget	Bell icon shows badge; alert shown in panel	Pass

Chapter 4

Project Screenshots

4.1 Landing Page

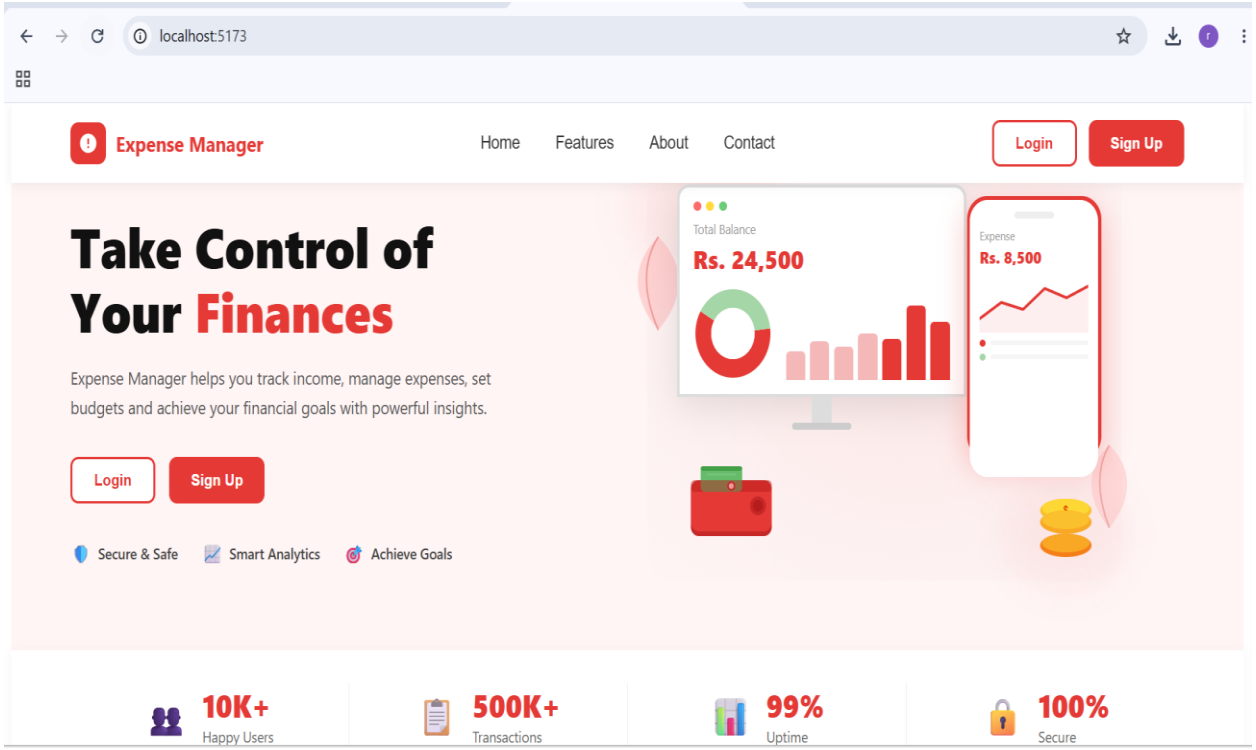


Figure 4.1: Landing Page – Hero Section

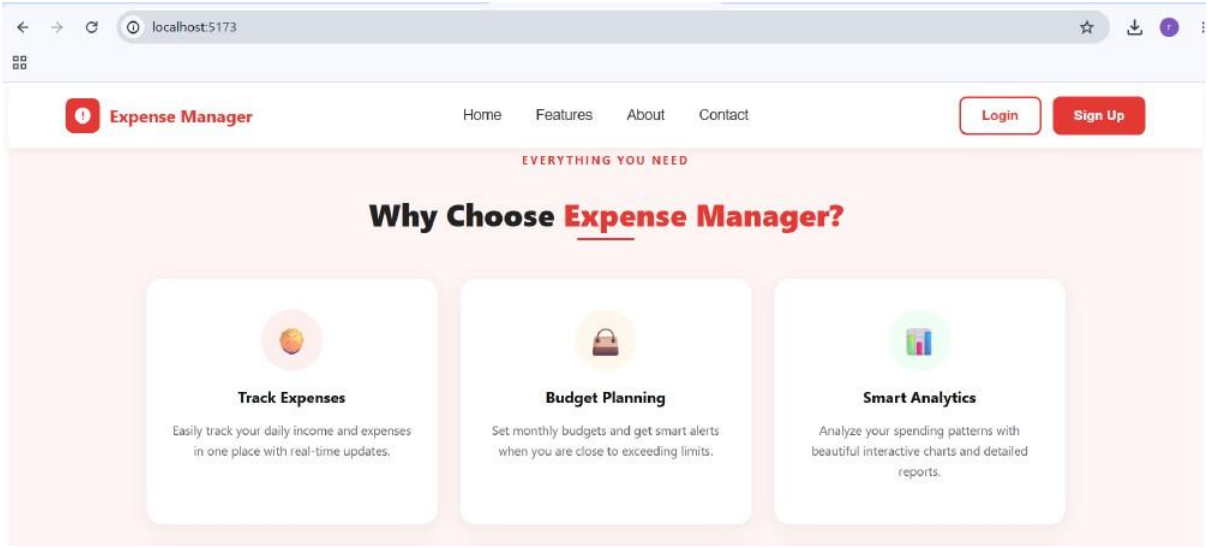


Figure 4.2: Landing Page – Features and About Section

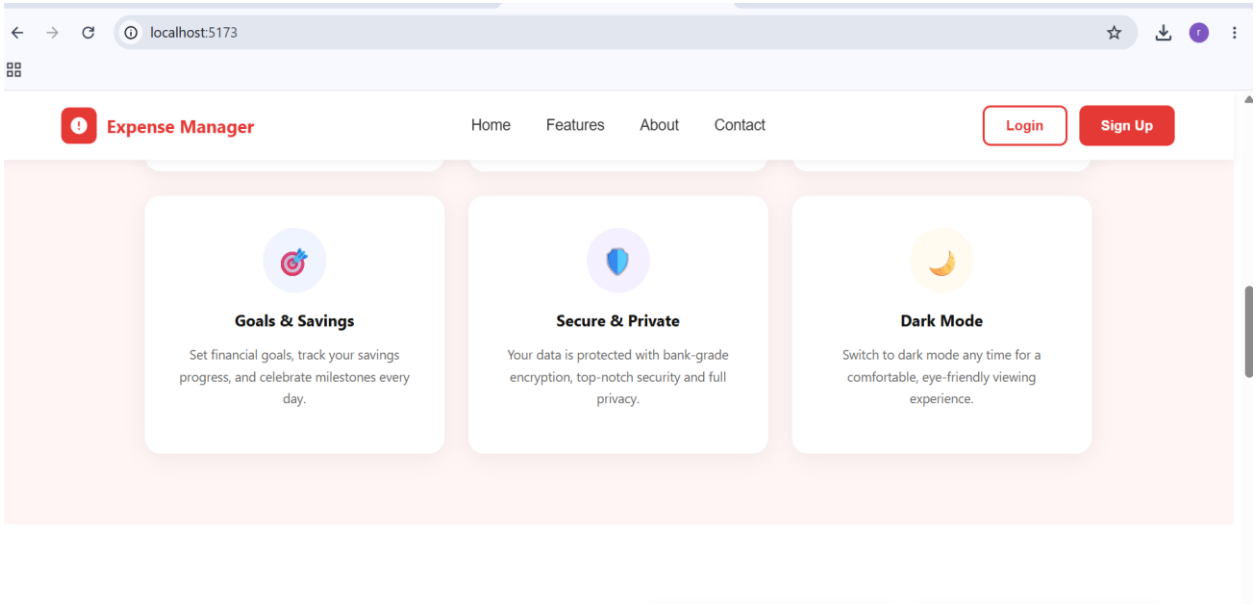


Figure 4.3: Landing Page – Goals, Secure, and Dark Mode Feature Cards

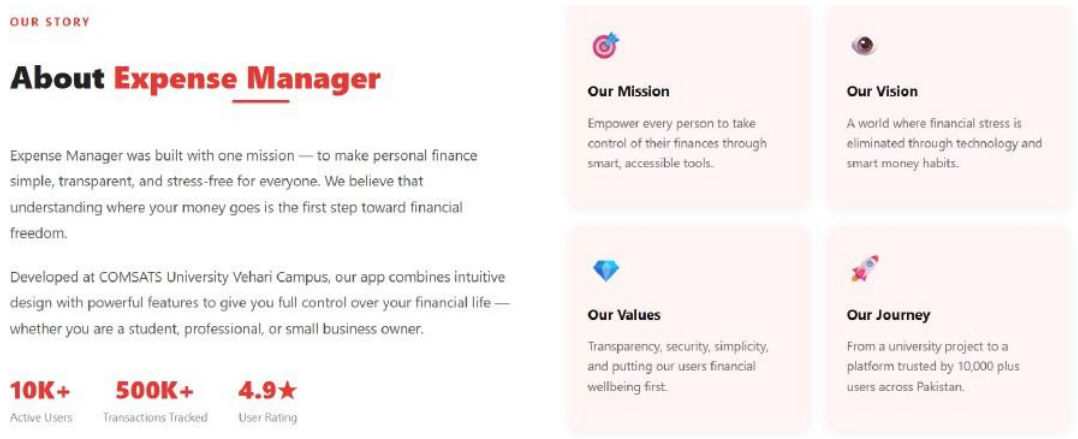


Figure 4.4: Landing Page – About Expense Manager Section

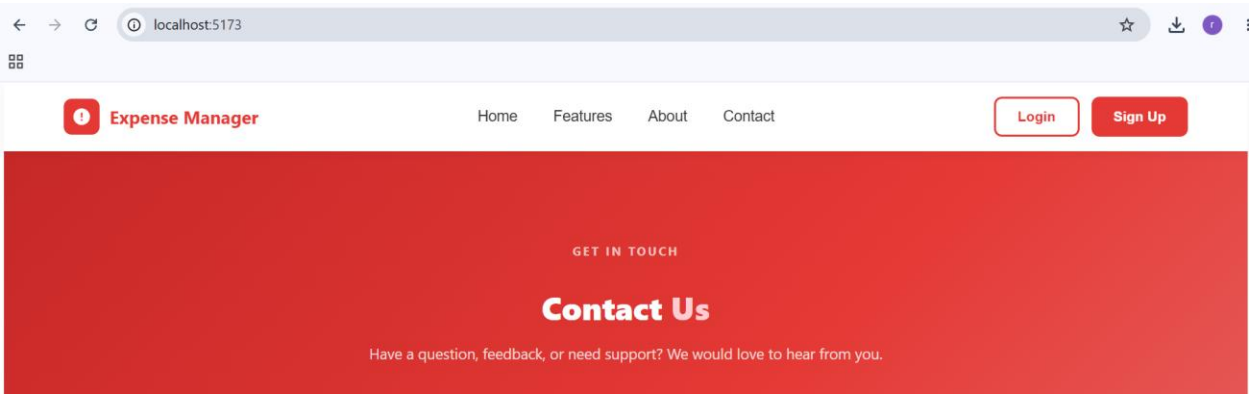


Figure 4.5: Landing Page – Contact Us Section Header

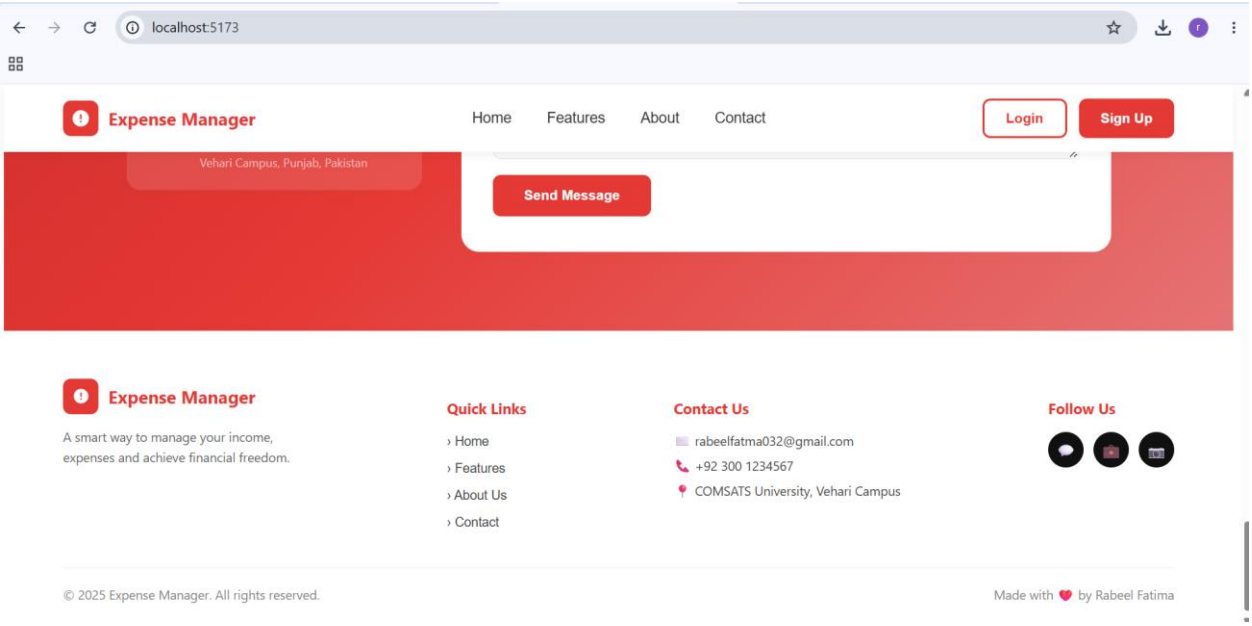


Figure 4.6: Landing Page – Contact Form and Footer

4.2 Authentication Pages

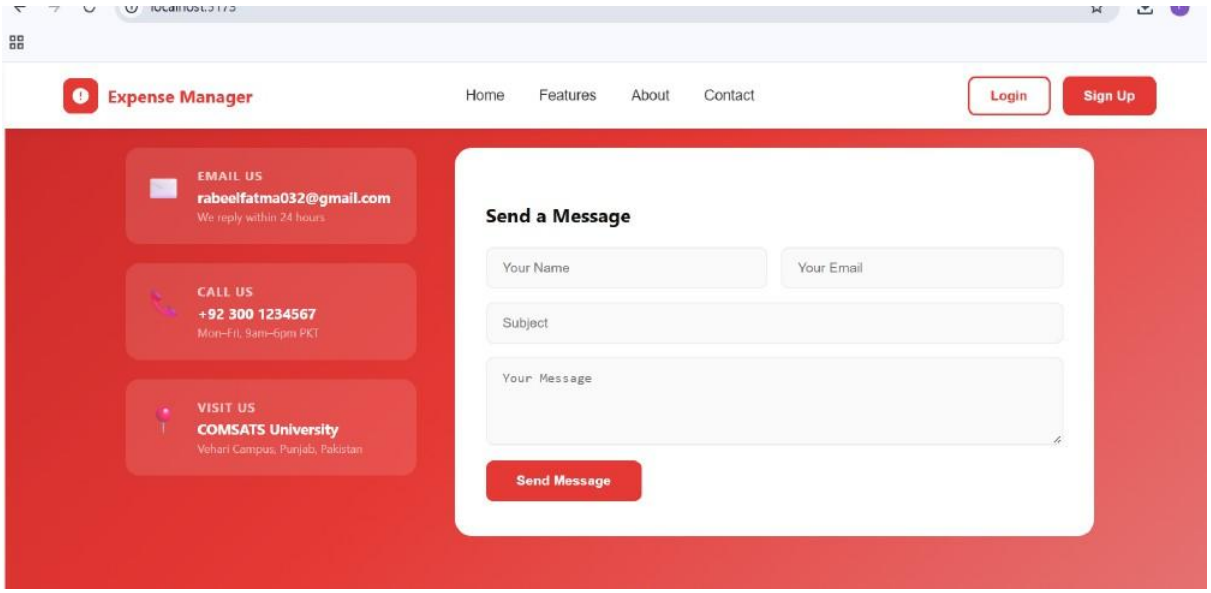


Figure 4.7: Signup Page – User Registration Form

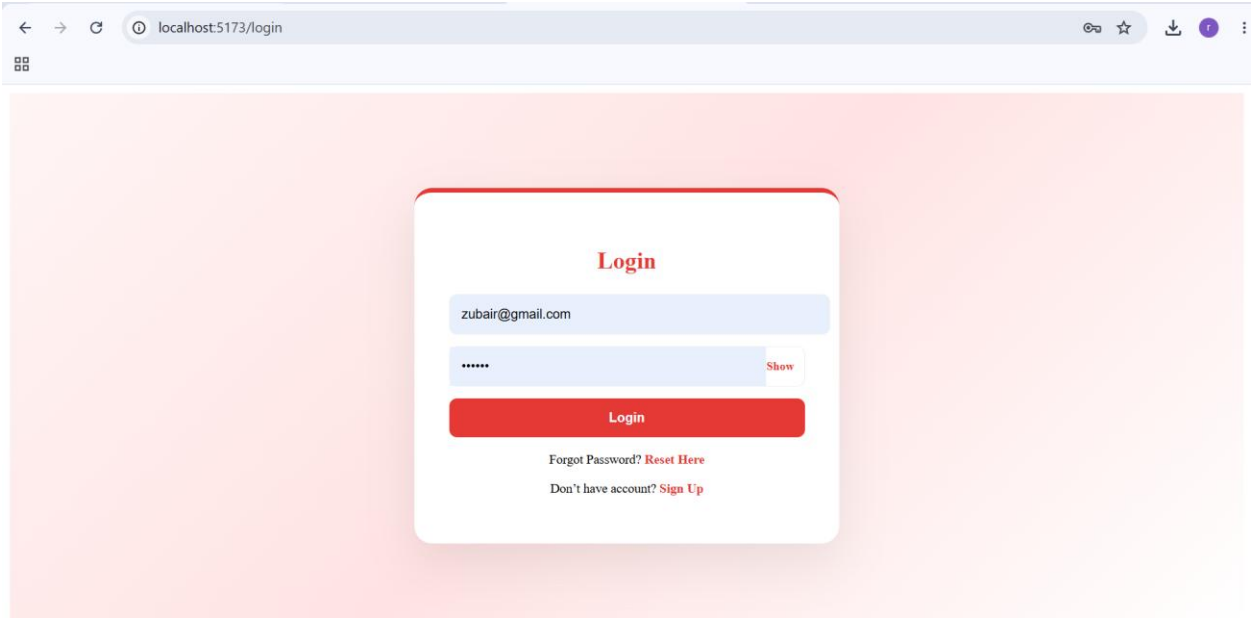


Figure 4.8: Login Page – User Authentication Form

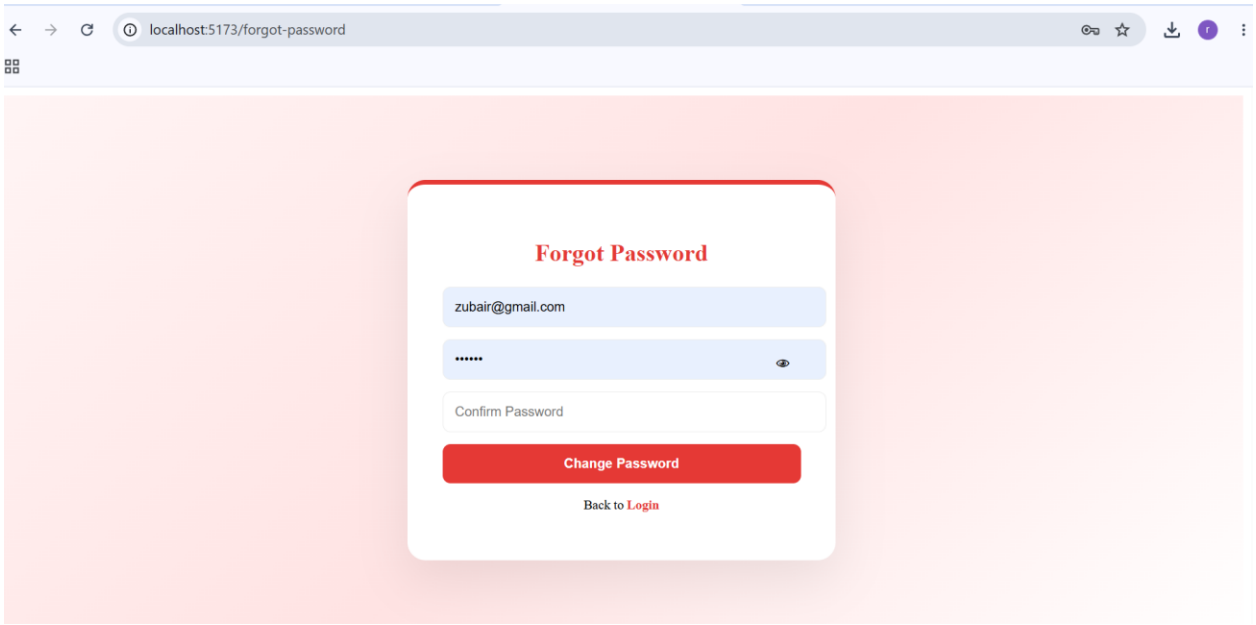


Figure 4.9: Forgot Password Page – Direct Password Reset

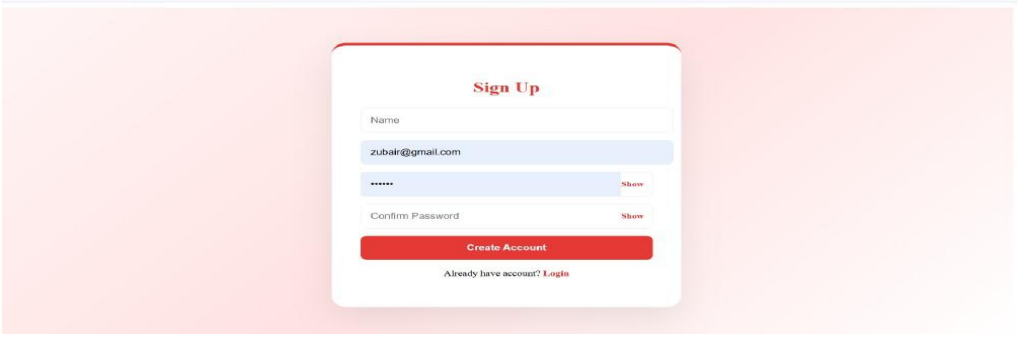


Figure 4.10: Singup

4.3 Dashboard

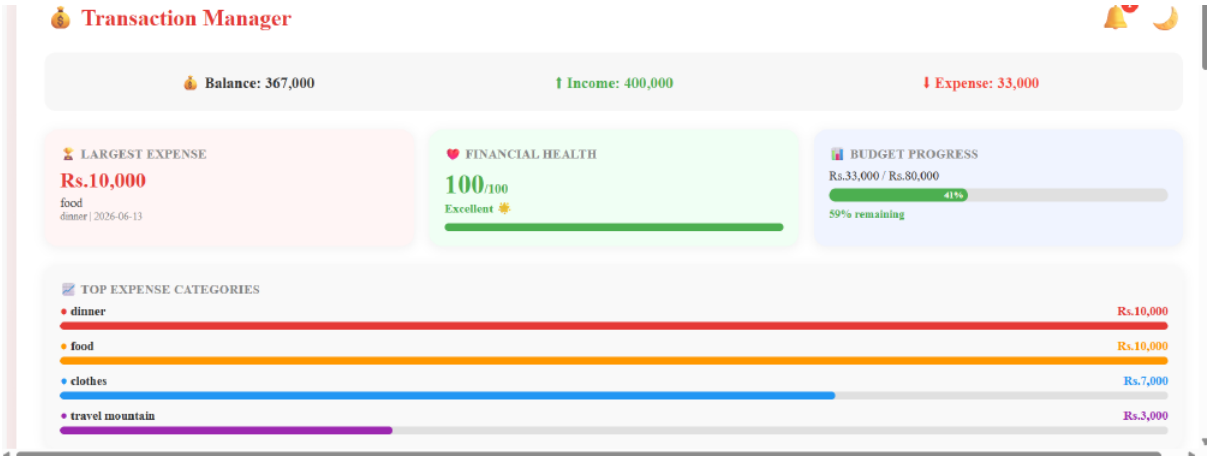


Figure 4.11: Dashboard – Financial Summary: Balance, Income, Expense Widgets

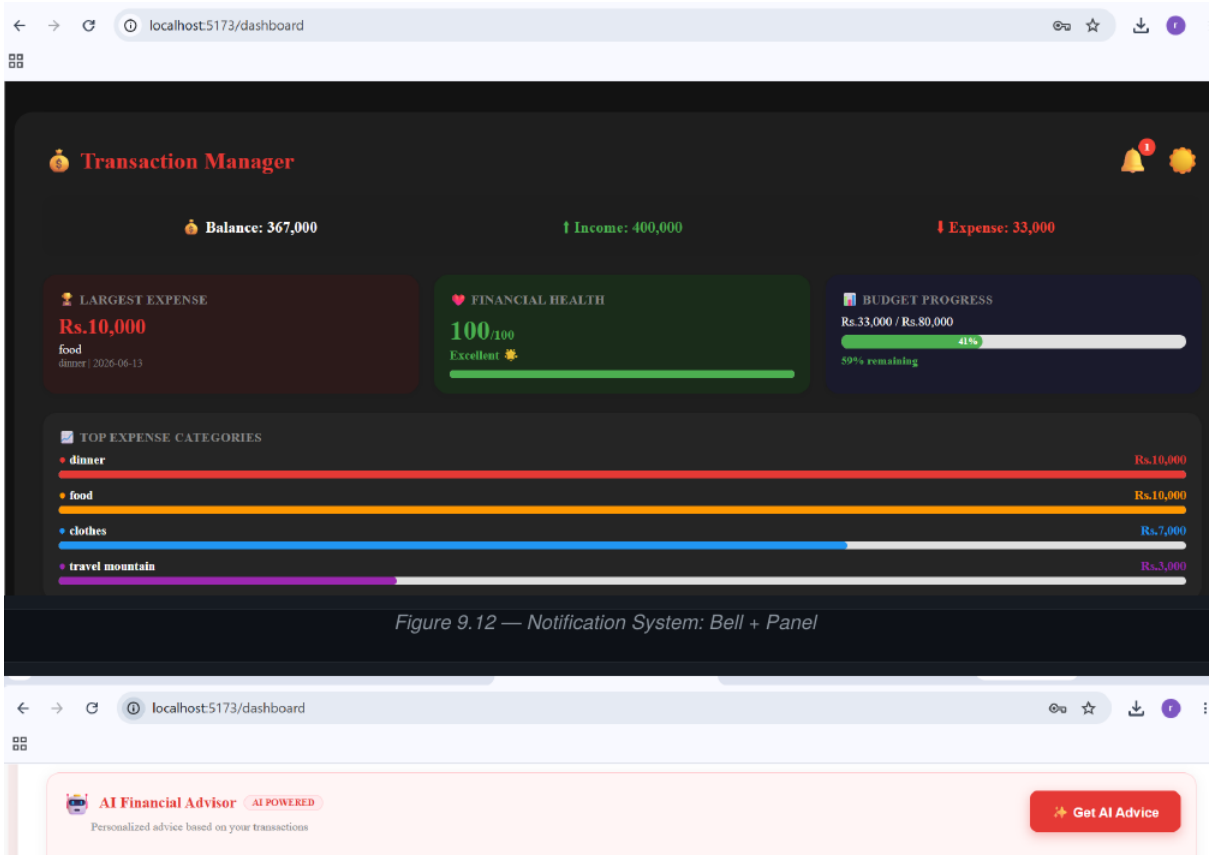


Figure 4.12: Dashboard – Notification System: Bell Icon and Notification Panel

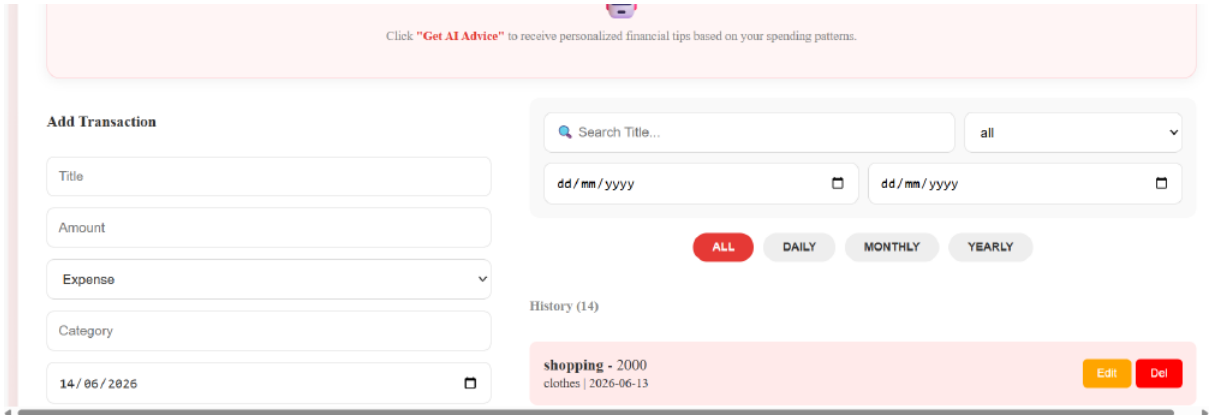


Figure 4.13: Dashboard – AI Financial Advisor and Transaction Form

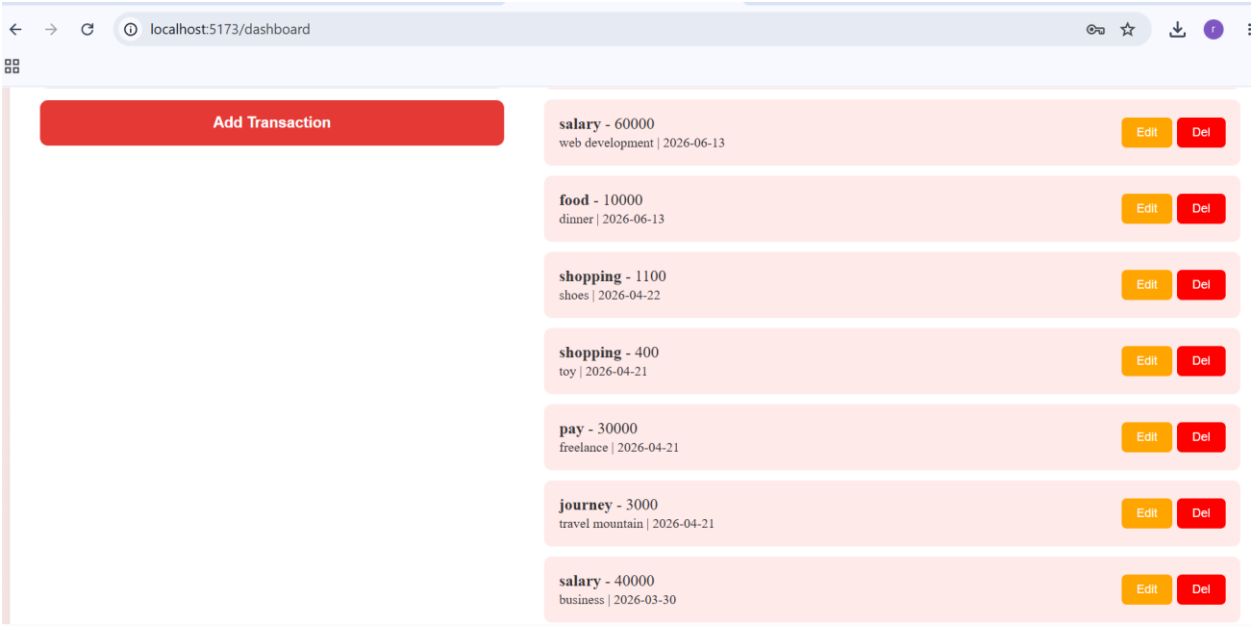


Figure 4.14: Dashboard – Transaction List with Edit and Delete Actions

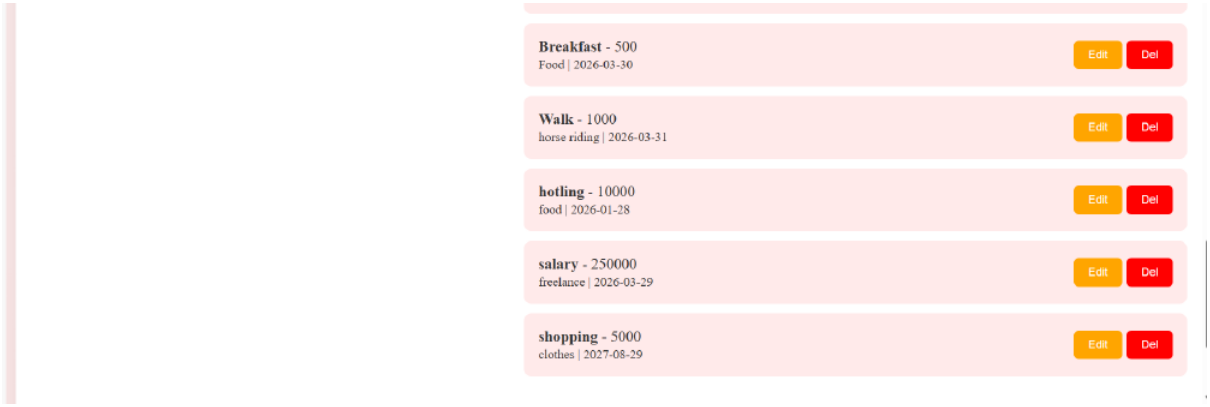


Figure 4.15: Dashboard – Full Transaction History List

Figure 4.16: Dashboard – Budget Limit Setting and Action Buttons (Analytics, Export PDF, Logout)

Figure 4.17: Dashboard – Budget Distribution Pie Chart and Income vs Expense Bar Chart

4.4 Analytics Page

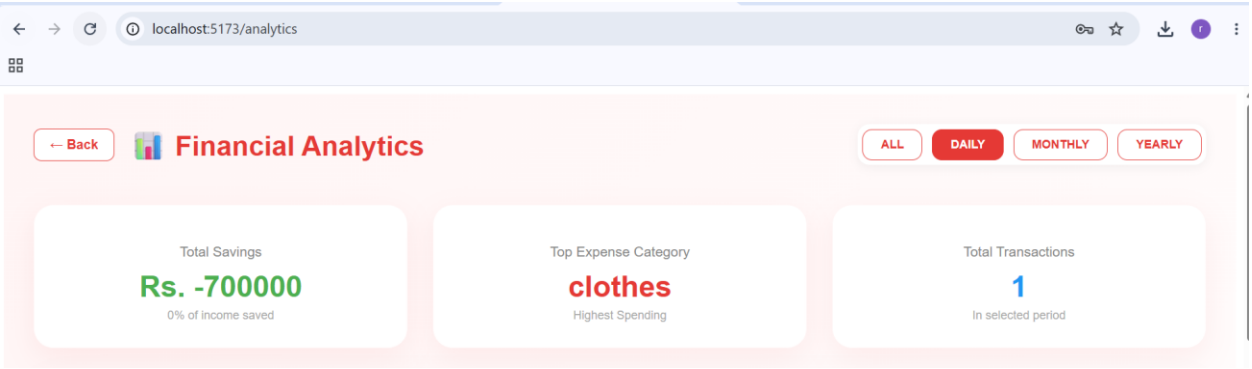


Figure 4.18: Analytics Page – Stats Cards: Total Savings, Top Category, Total Transactions

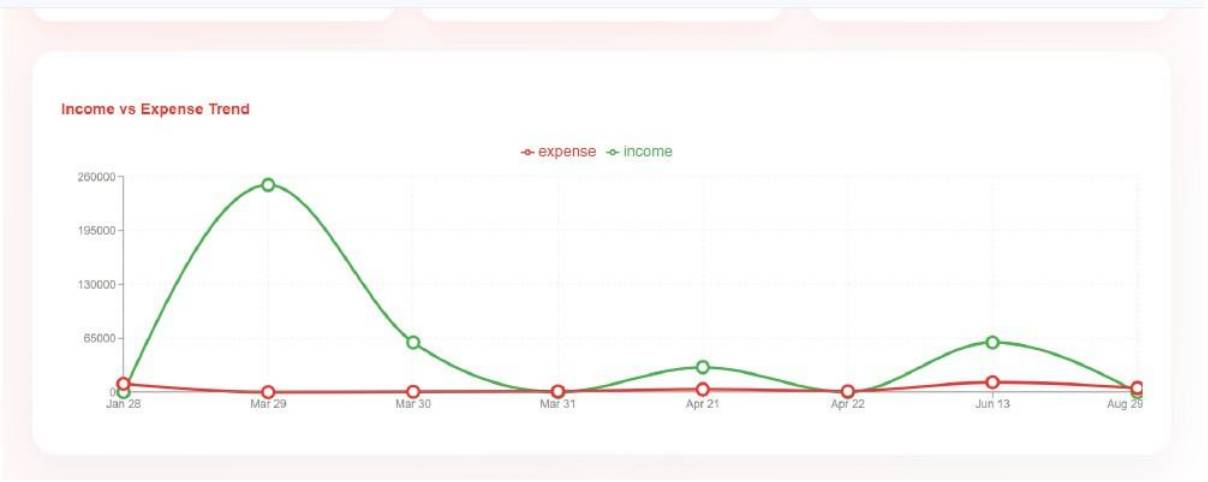


Figure 4.19: Analytics Page – Income vs Expense Line Chart (All Time View)

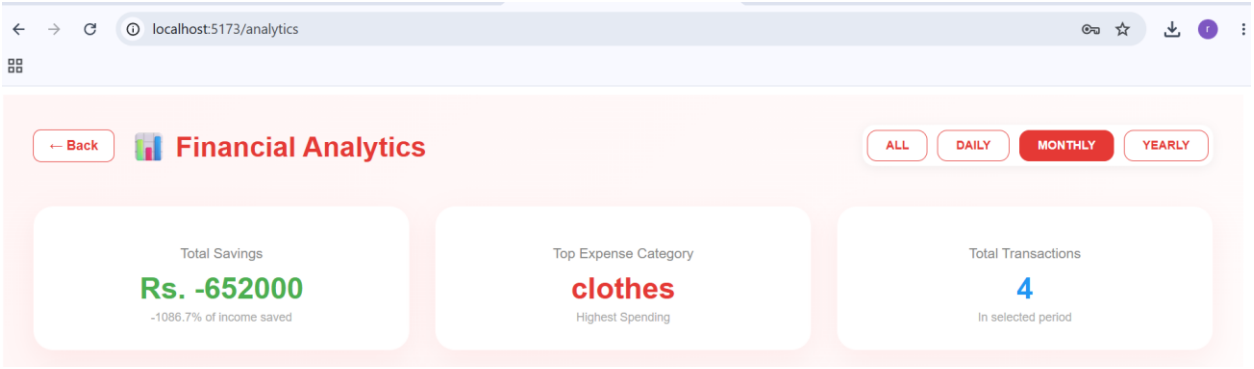


Figure 4.20: Analytics Page – Daily Filter: Stats Cards View

Figure 4.21: Analytics Page – Income vs Expense Trend (Monthly Filter)

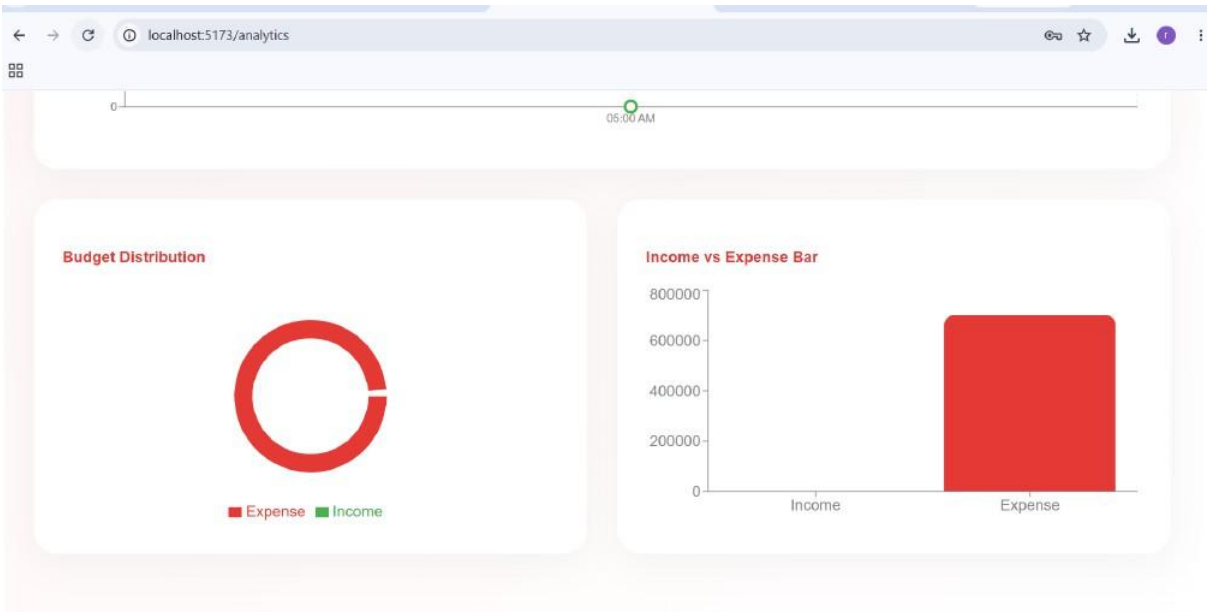


Figure 4.22: Analytics Page – Budget Distribution Pie Chart and Income vs Expense Bar Chart

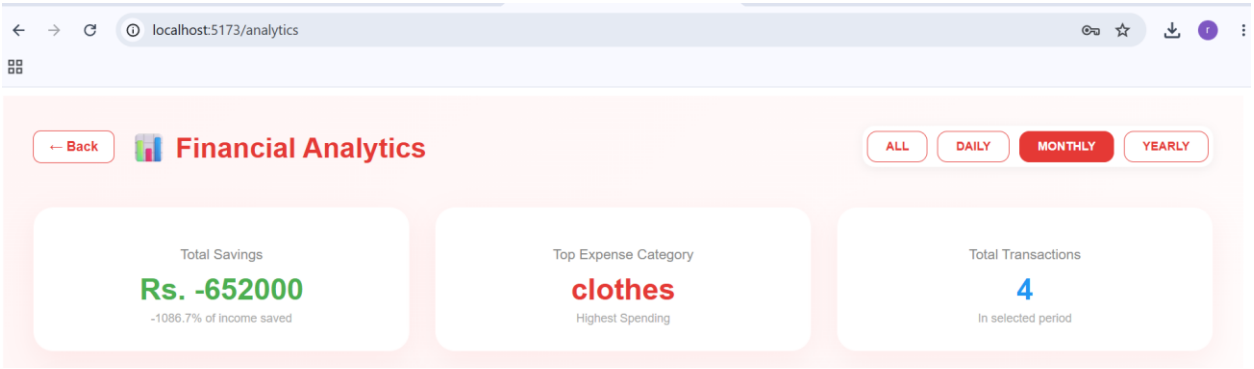


Figure 4.23: Analytics Page – Monthly Stats: Negative Savings Scenario

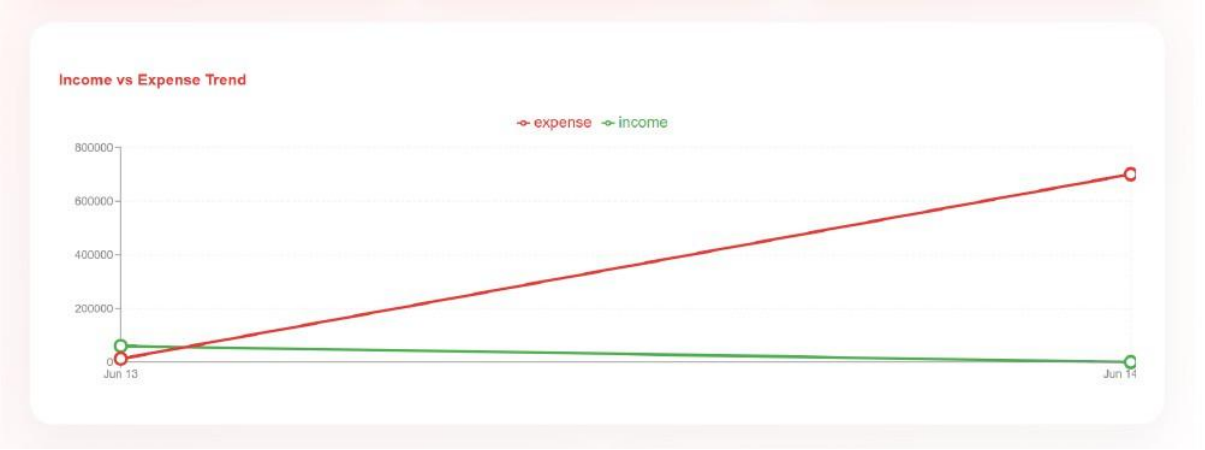


Figure 4.24: Analytics Page – Monthly Line Chart: Income vs Expense Trend

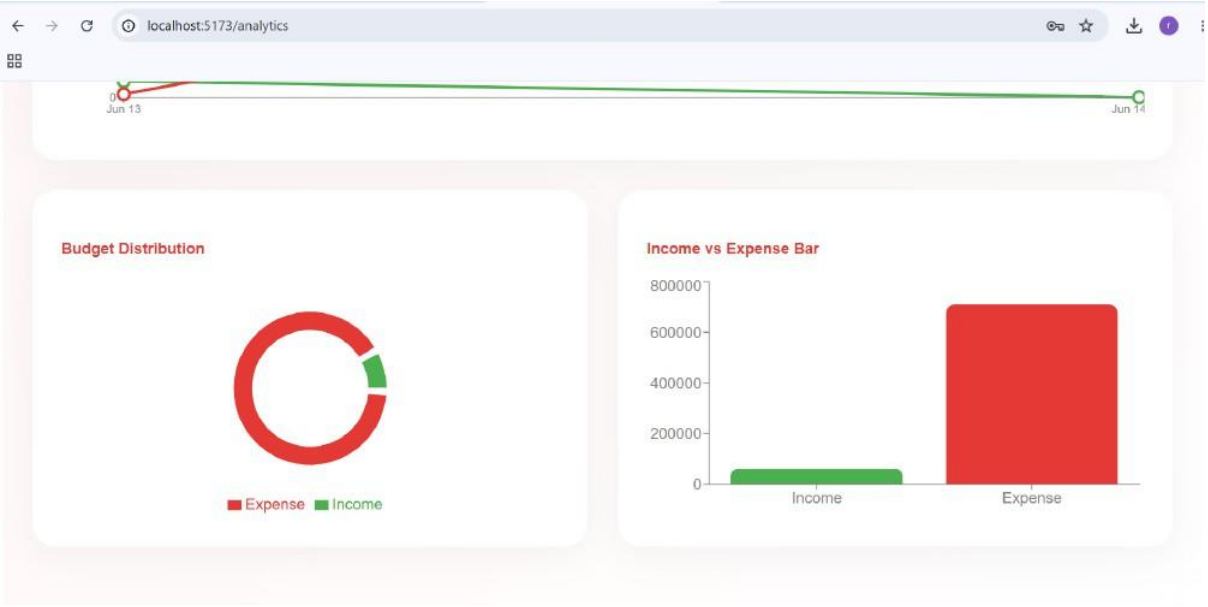


Figure 4.25: Analytics Page – Budget Pie Chart and Income vs Expense Bar (Monthly)

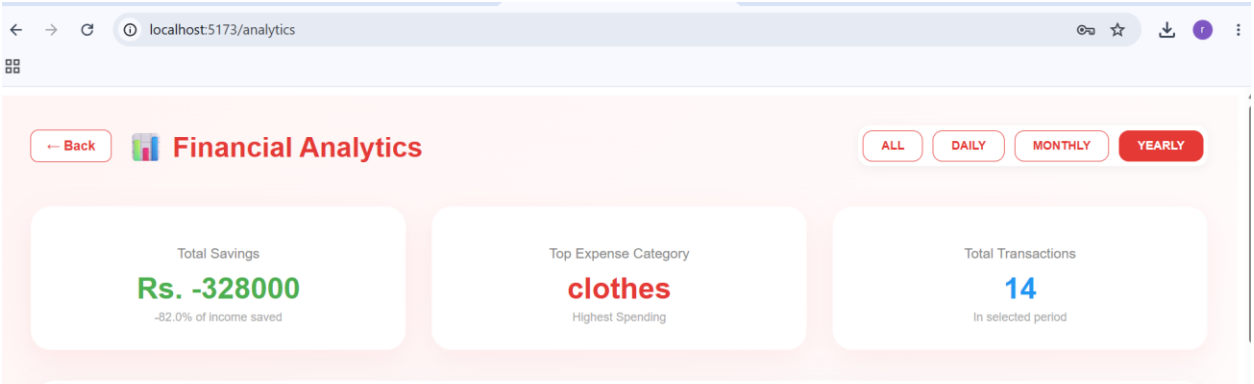


Figure 4.26: Analytics Page – Yearly Filter Stats Cards

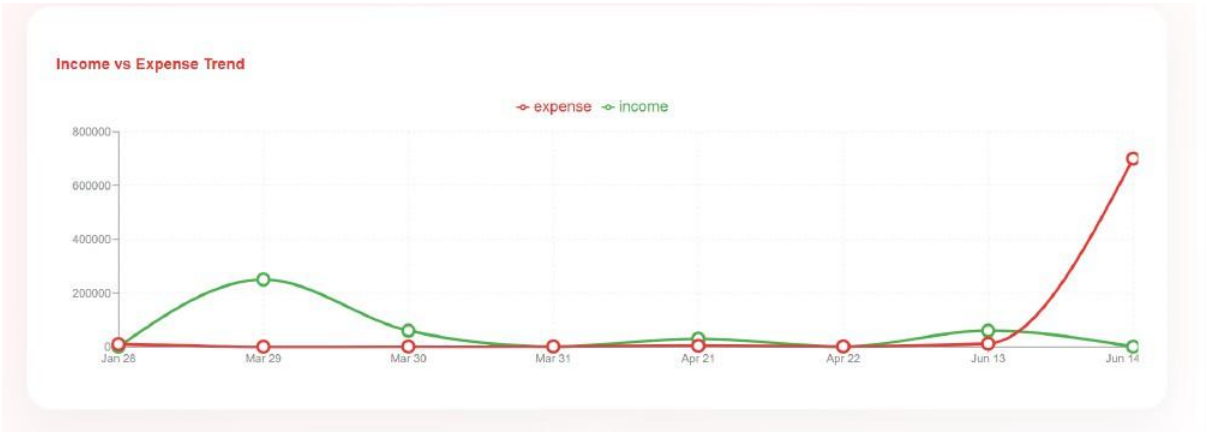


Figure 4.27: Analytics Page – Yearly Income vs Expense Line Chart

Figure 4.28: Analytics Page – Yearly Budget Distribution and Bar Chart

4.5 Database (MongoDB)

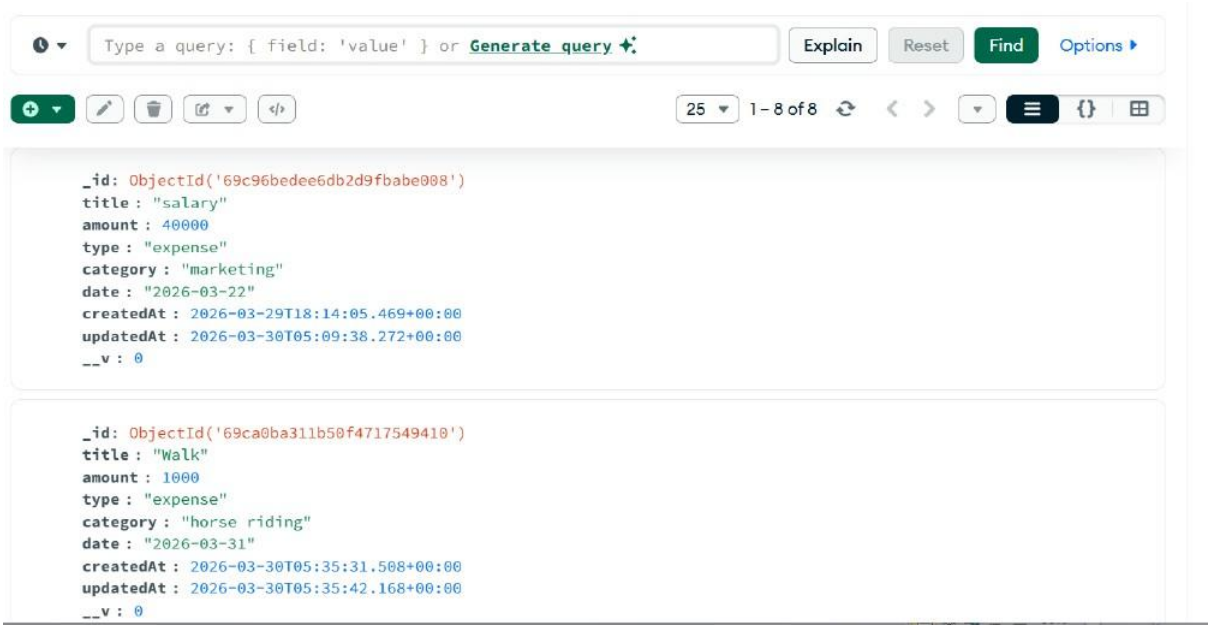


Figure 4.29: MongoDB – Transactions Collection Document Structure

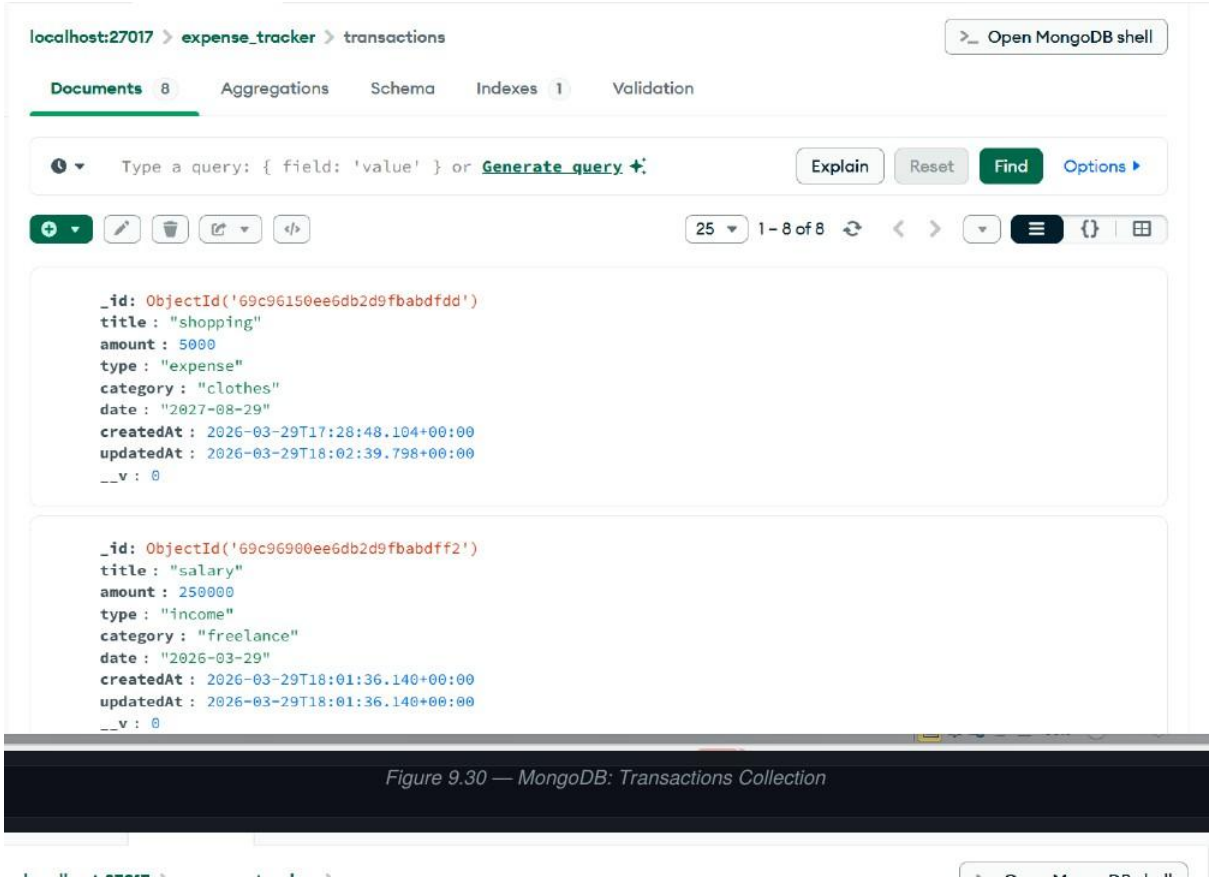


Figure 9.30 — MongoDB: Transactions Collection

Figure 4.30: MongoDB – Transactions Collection (8 Documents View)

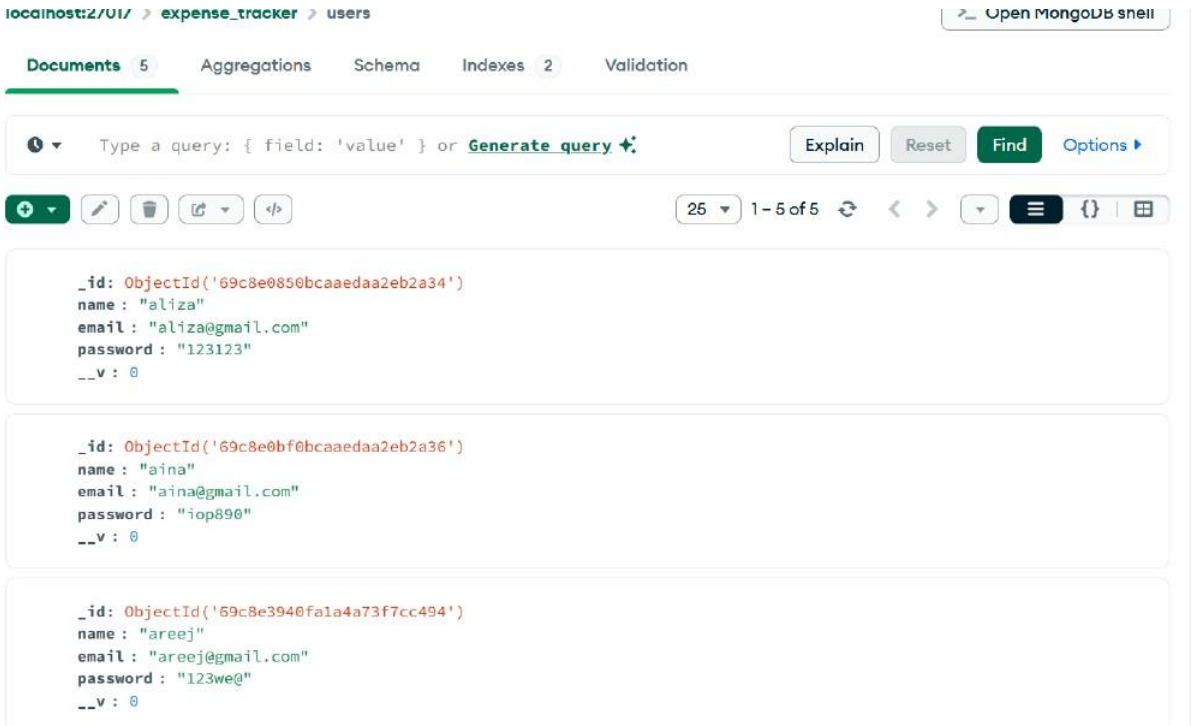


Figure 4.31: MongoDB – Users Collection (5 Documents View)

Figure 4.32: MongoDB – Users Collection Document Structure (with Hashed Password)

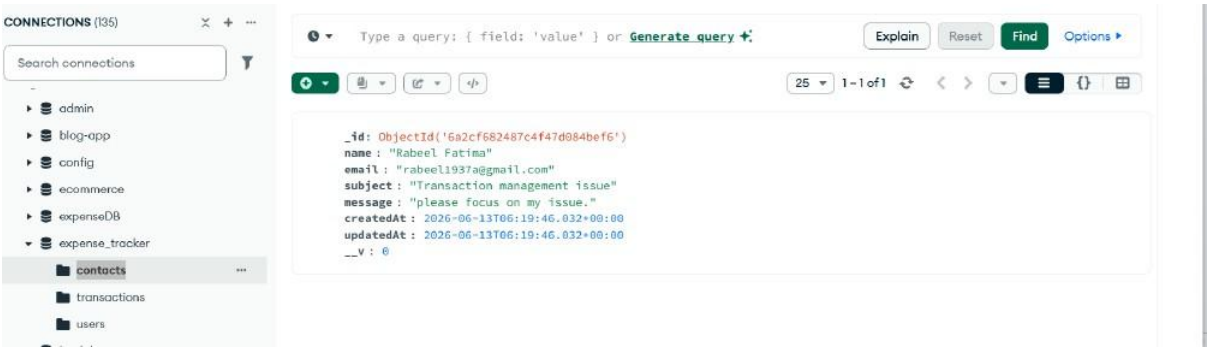
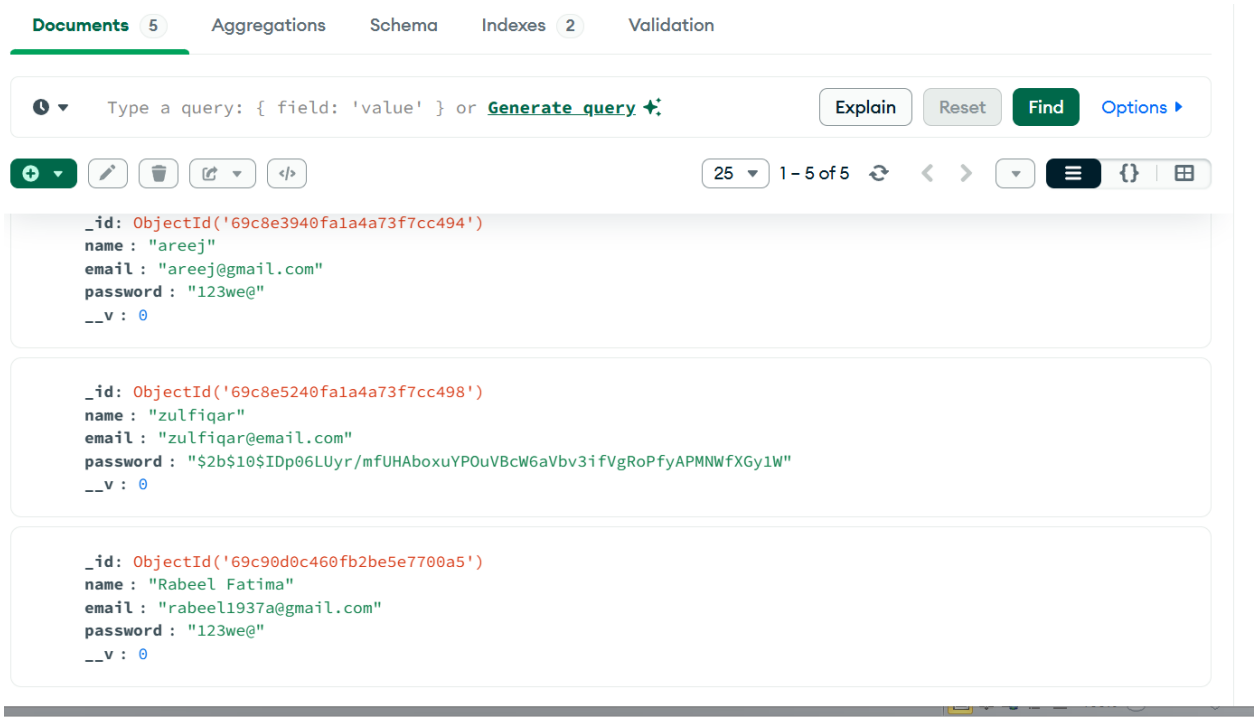


Figure 4.33: MongoDB – Contacts Collection with Contact Form Submission



4.6 Testing Screenshots

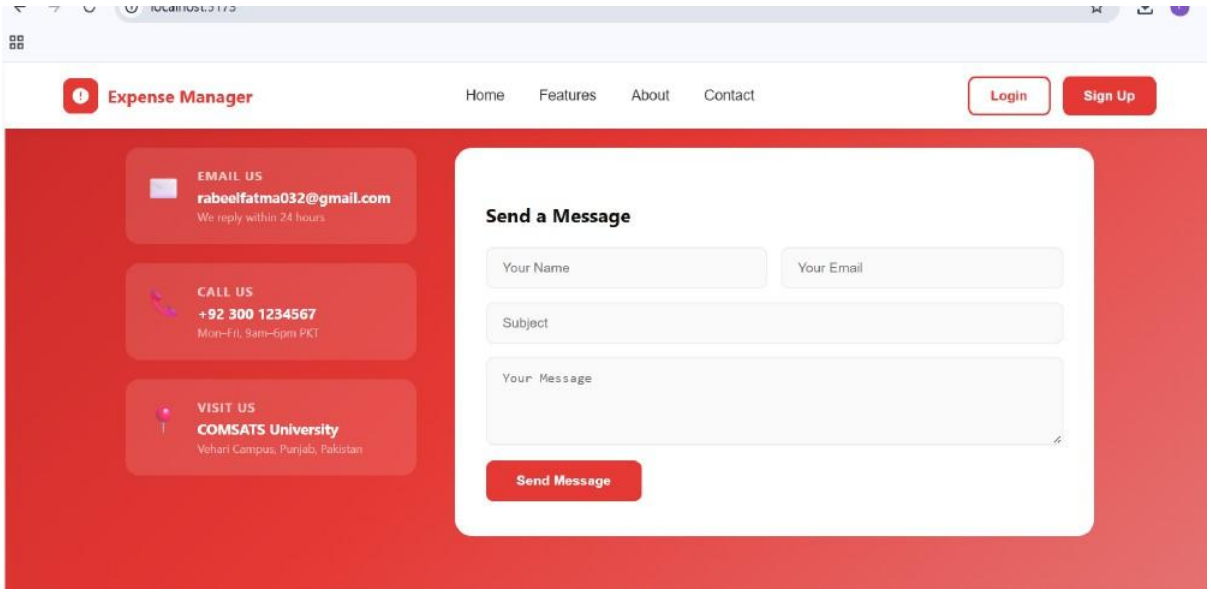


Figure 4.34: TC-1.1a – Signup Page with Registration Form

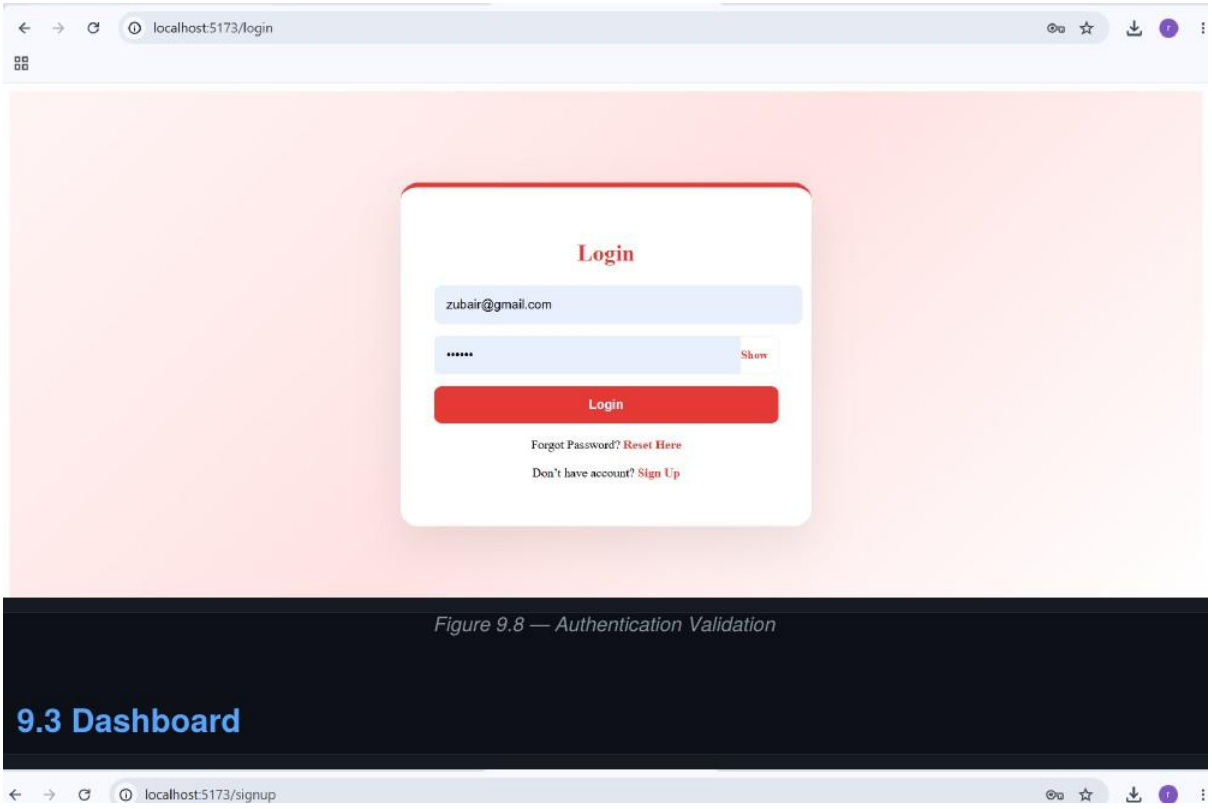


Figure 4.35: TC-1.2a – Login Page Showing Valid Credentials Entry

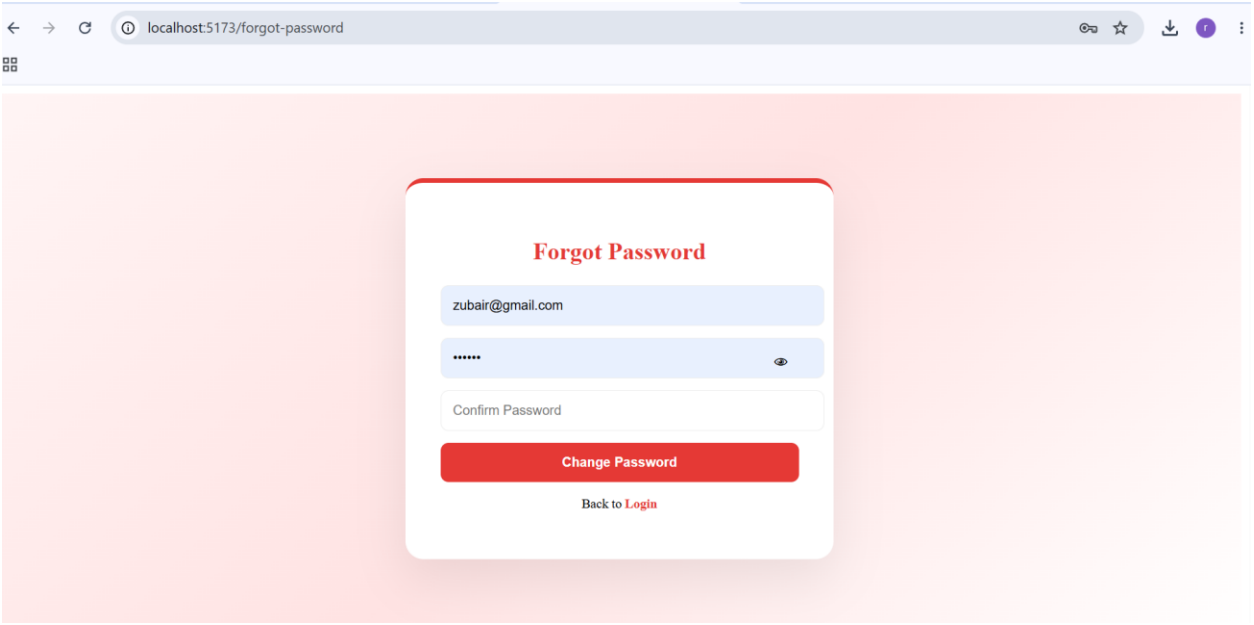


Figure 4.36: TC-1.3b – Forgot Password Page with Change Password Form

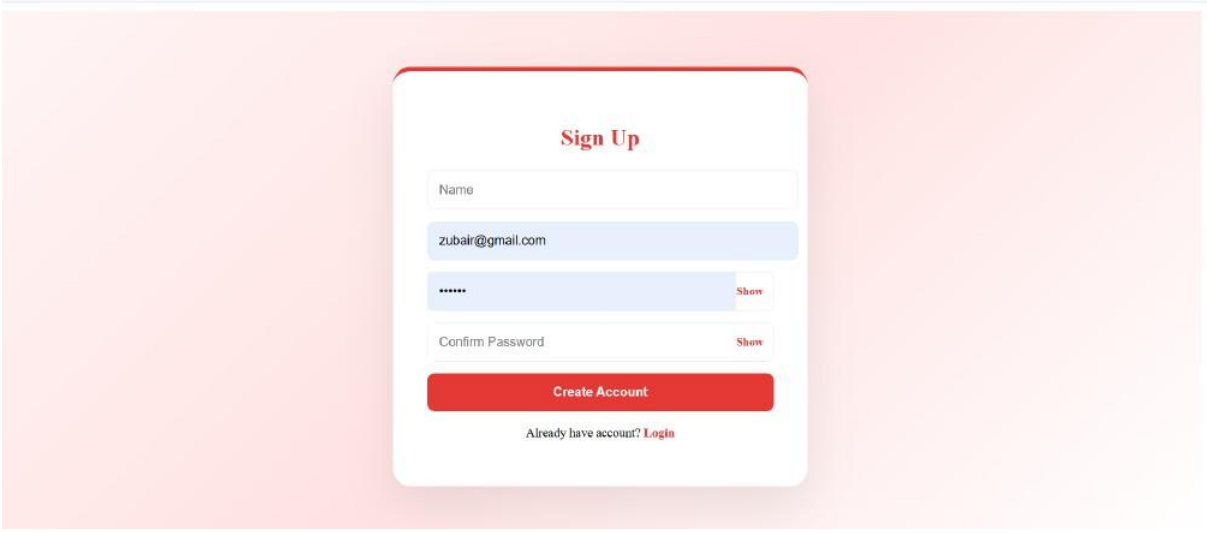


Figure 4.37: TC-2.1a – Dashboard Showing Live Balance, Income, and Expense Summary

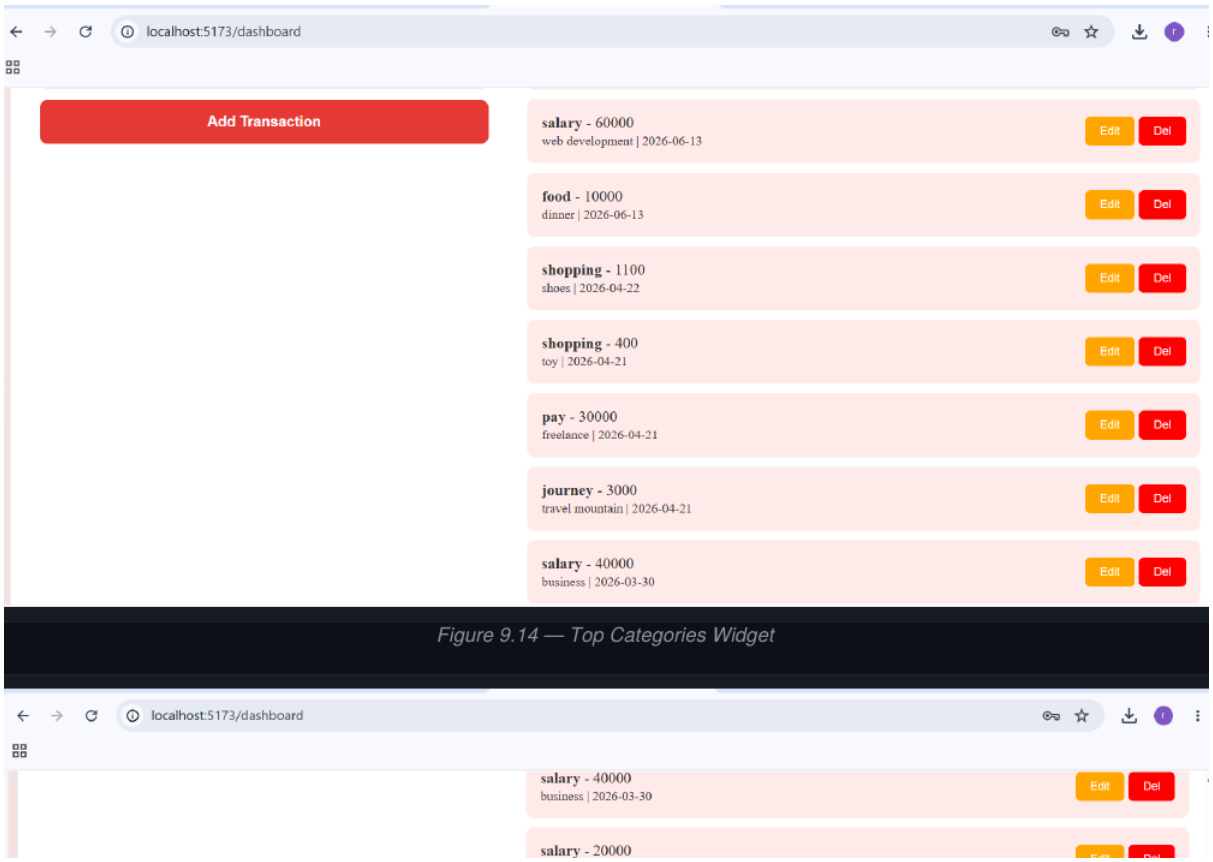


Figure 4.38: TC-2.2a – Transaction List with Edit and Delete Buttons

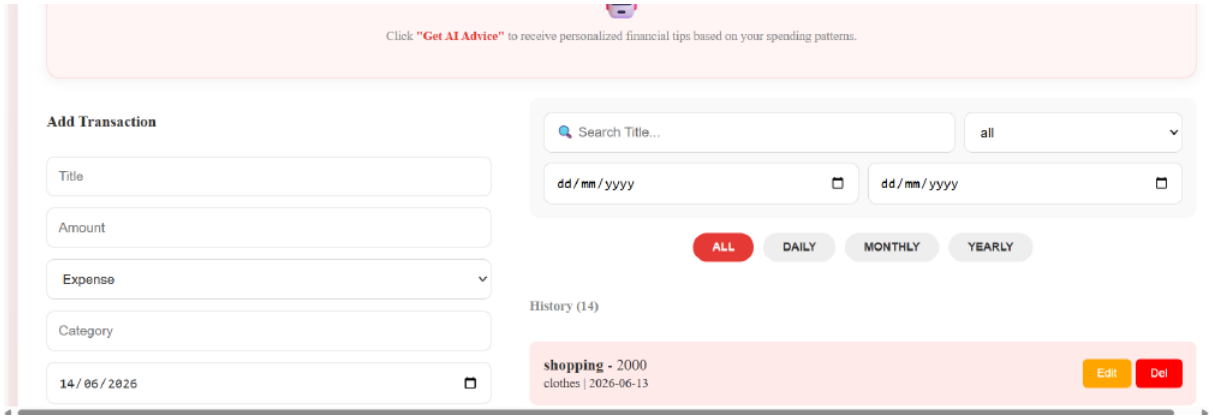


Figure 4.39: TC-4.1a – AI Financial Advisor Section with Get AI Advice Button

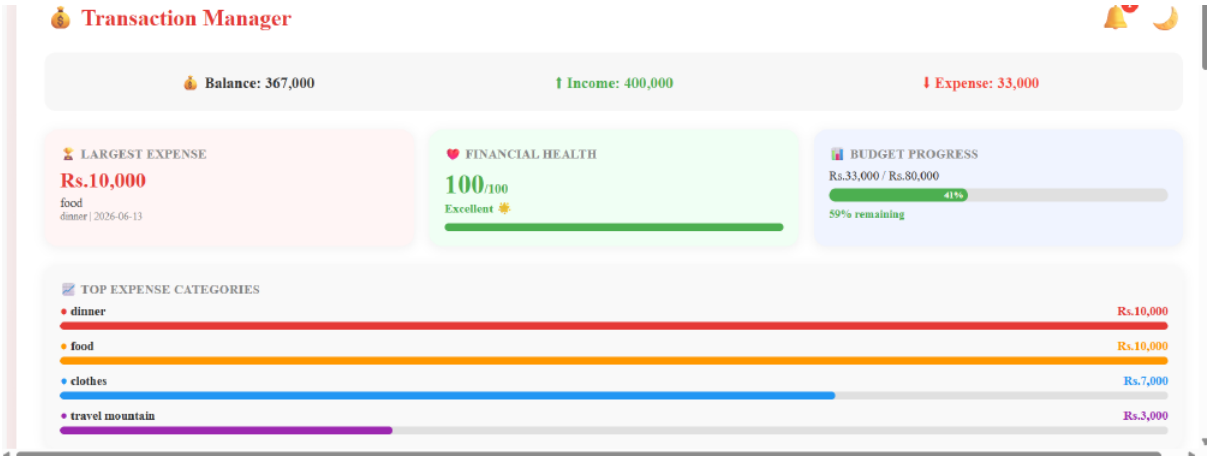


Figure 4.40: TC-5.1b – Largest Expense, Financial Health Score, and Budget Progress Widgets

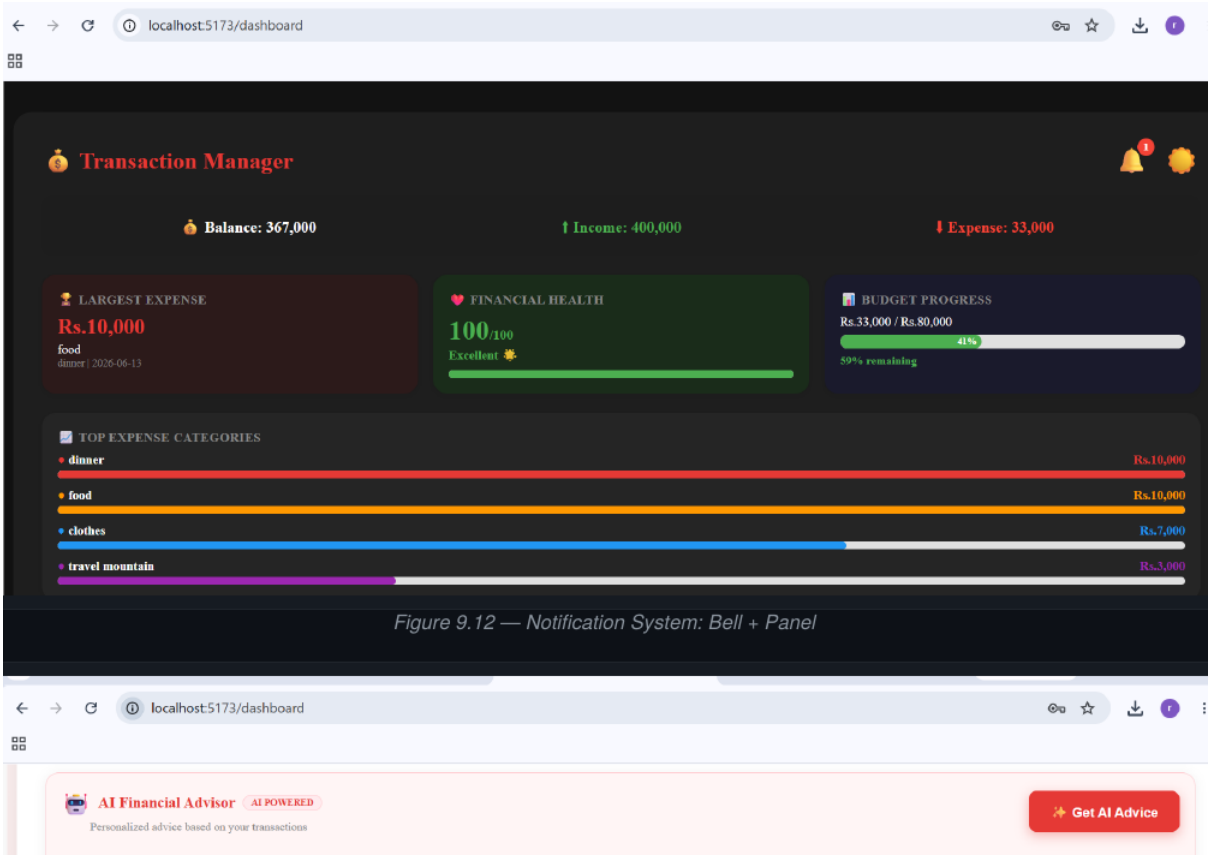


Figure 4.41: TC-5.1c – Dashboard with Top Expense Categories and Notification Bell

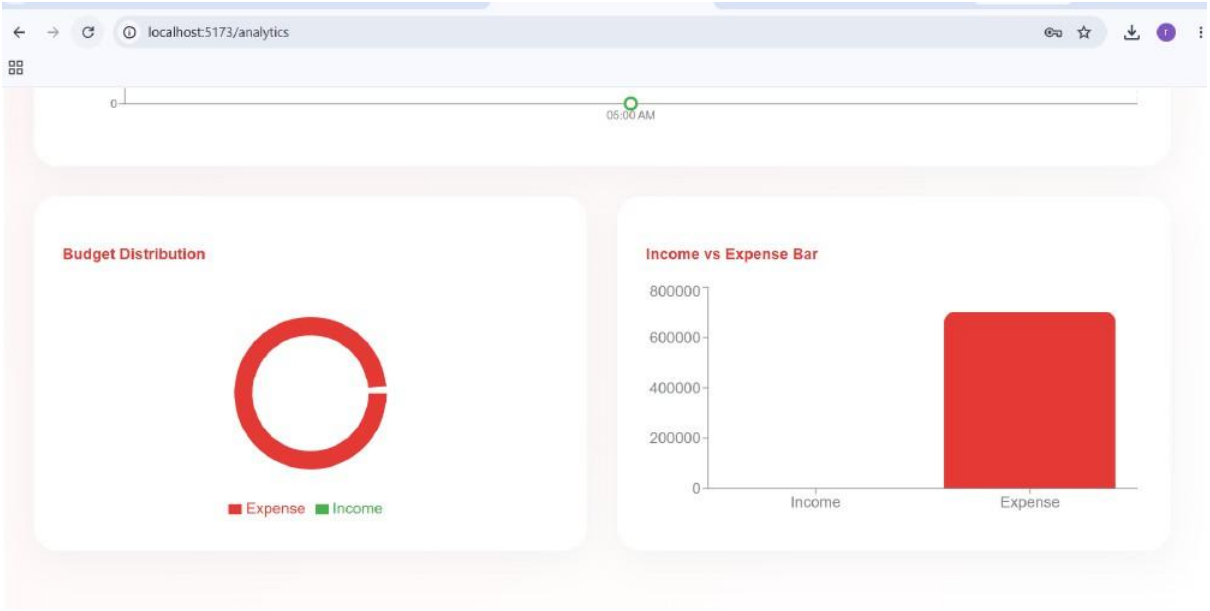


Figure 4.42: TC-3.1b and TC-3.1c – Budget Distribution Pie Chart and Income vs Expense Bar Chart

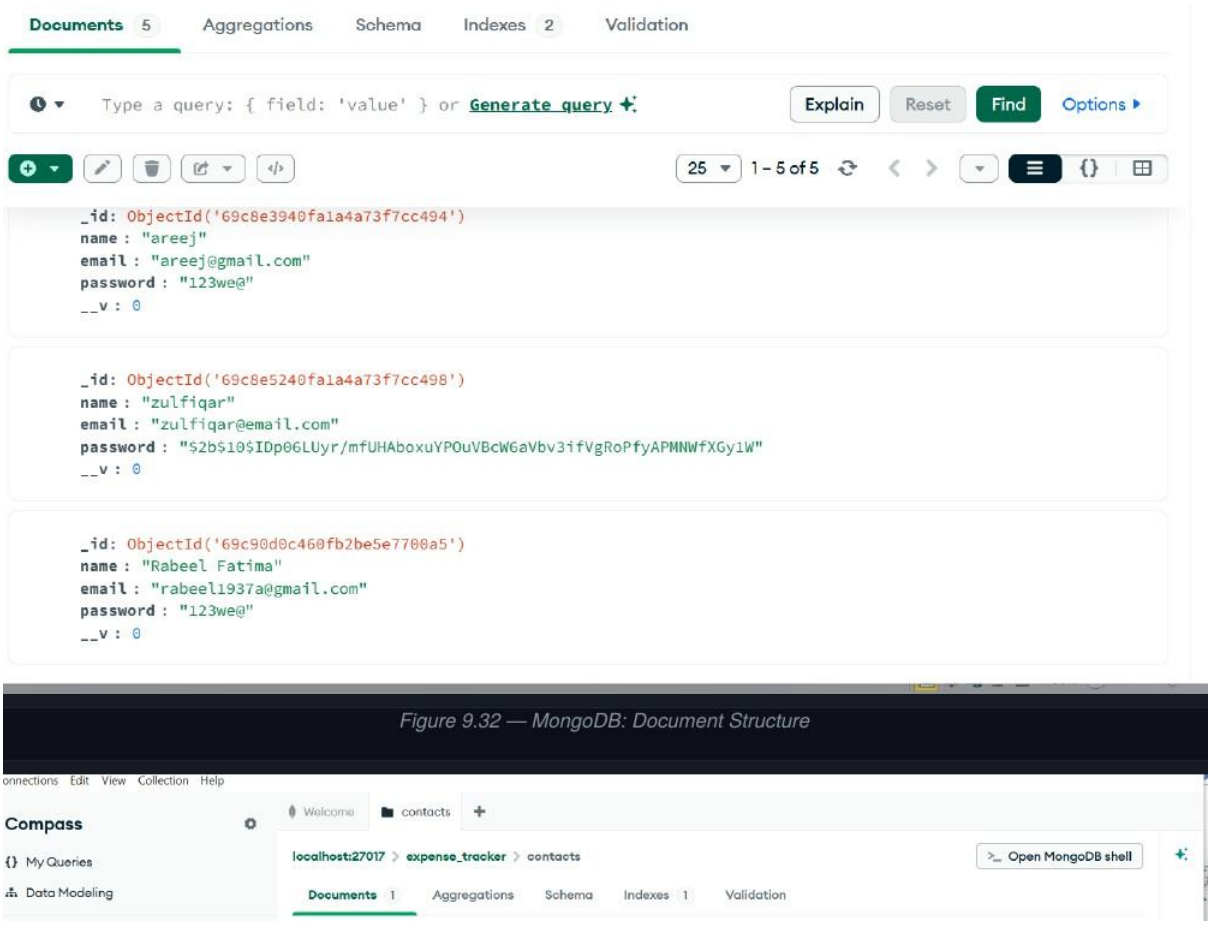


Figure 4.43: TC-1.4a – MongoDB Users Collection Showing Bcrypt Hashed Password

Bibliography

1. Google Gemini AI API Documentation. https://ai.google.dev, 2024.
2. JSON Web Tokens – Introduction. https://jwt.io/introduction, 2024.
3. MongoDB – The Developer Data Platform. https://www.mongodb.com, 2024.
4. Node.js – JavaScript Runtime Built on Chrome's V8 JavaScript Engine. https://nodejs.org, 2024.
5. React – A JavaScript Library for Building User Interfaces. https://reactjs.org, 2024.

Video:

expense tracker.mp4